Cross-National Prediction of COVID-19 Protective Behaviour Before and After Mask Mandates

Final Report

Zixuan Zhao

2026-07-25

1 Declaration

In submitting this work I am indicating that I have read the University's Academic Integrity Policy. I declare that all material in this assessment is my own work except where there is clear acknowledgement and reference to the work of others. I give permission for this work to be reproduced and submitted to other academic staff for educational purposes.

2 Abstract

This report examines whether the predictors of self-reported COVID-19 protective behaviour changed after the onset of sustained face-mask mandates and whether these patterns were shared across countries. Repeated cross-sectional YouGov survey data were linked to subnational policy records from the Oxford COVID-19 Government Response Tracker for Australia, Brazil, Canada, India, the United Kingdom and the United States. Two binary outcomes were analysed: frequent or consistent face-mask wearing and overall protective behaviour. Separate before- and after-mandate models were fitted for each country and outcome using Random Forest and XGBoost, producing 24 modelling tasks. Predictive performance was evaluated using held-out ROC AUC, and model-based feature importance was used to compare the leading predictors.

XGBoost achieved the higher test AUC in 19 of the 24 tasks, although its average performance was only slightly higher than Random Forest. Face-mask wearing was most consistently predicted by engagement in other protective behaviours, whereas overall protective behaviour was most consistently predicted by willingness to self-isolate. Risk perception, symptom-related isolation, physical contact, survey timing, employment and confidence-related variables also contributed. Predictor importance changed between policy periods in every country, but the direction and structure of these changes differed. The results indicate a shared behavioural core alongside country- and period-specific predictive patterns.

3 Introduction

Protective behaviours such as face-mask wearing, physical distancing, hand hygiene and self-isolation were central to reducing infection risk during the COVID-19 pandemic (Lisi et al. 2021, 1–2). Understanding why individuals adopted these behaviours is important because public-health measures depend not only on the existence of policy, but also on how people respond within different social and institutional settings. Previous research has associated protective behaviour with demographic characteristics, perceived risk, trust, psychological factors and national context (Galasso et al. 2020, 27285–87; Roma et al. 2020, 2–6; Van Lissa et al. 2022, 2–8; Taye et al. 2023, 11–12). These relationships may also change over time as policies, public information, infection levels and social expectations develop.

A mask mandate is a government policy requiring people to wear face coverings in specified public settings. The introduction of such mandates provides a useful policy boundary for examining whether the factors associated with protective behaviour change over time. Earlier research in Australia found that the leading predictors of mask wearing and broader protective behaviour differed before and after mandate onset (Ryan et al. 2025, 2–4, 8–11). However, countries differ in survey timing, population characteristics, institutions and policy histories. A cross-national comparison using a common analytical procedure can therefore distinguish recurring predictive patterns from those that depend more strongly on national context.

This report analyses repeated cross-sectional survey responses from the Imperial College London/YouGov COVID-19 Behaviour Tracker and links them to face-covering policy records from the Oxford COVID-19 Government Response Tracker (Imperial College London and YouGov 2022; Oxford COVID-19 Government Response Tracker 2023). Six countries are examined: Australia, Brazil, Canada, India, the United Kingdom and the United States. In this report, compliance refers to frequent or consistent self-reported performance of a protective behaviour; it does not represent direct observation of legal compliance. Two outcomes are considered: face-mask wearing and overall protective behaviour. Their definitions are held constant across countries. Each national model contains a shared core of predictors together with additional variables retained from the corresponding country data.

Survey responses are compared across the periods before and after the onset of sustained mask mandates in each country. The analysis considers whether the factors associated with face-mask wearing and broader protective behaviour remain similar or change across policy periods and national settings.

The analysis addresses three research questions:

1. Within each country, do the key predictors of compliance with COVID-19 protective behaviours change before and after mask mandates?
2. Are these changes consistent across countries, or do they vary?
3. Is there a set of common cross-national predictors alongside country-specific predictors?

By comparing countries and policy periods, we hope to identify behavioural and attitudinal factors that recur across different settings, while also highlighting predictors that depend more strongly on national context. We also hope to clarify whether protective behaviour can be understood through a broadly shared predictive structure or whether its leading predictors vary mainly by country and policy period.

4 Background

This report analyses self-reported behaviours from repeated survey waves and links each response to the policy context at the survey date. The main terms used throughout the report are defined in Table 1.

Term	Meaning in this report
Protective behaviour	Self-reported actions intended to reduce infection risk, such as mask wearing, physical distancing, hand hygiene, reducing close contacts and self-isolation.
Compliance	Frequent or consistent self-reported performance of a protective behaviour. It is not direct observation of legal compliance.
Policy period	Whether a survey response was observed before or after sustained mask-mandate onset.
Mask mandate	A policy context identified from the OxCGRT face-covering indicator, rather than a direct measure of enforcement or individual behaviour.
Feature importance	A model-based measure of predictive contribution within a specific country, policy period, outcome and model. It is not interpreted as a causal effect.

Table 1: Key terms used in this report.

Studies of COVID-19 protective behaviour have identified demographic, psychological and social predictors. Pandemic attitudes and reported behaviour differed by gender and national setting (Galasso et al. 2020, 27285–87; Haischer et al. 2020, 1–2). Perceived risk, trust and psychological characteristics were also associated with compliance (Roma et al. 2020, 2–6; Taye et al. 2023, 11–12), while the relative importance of predictors varied across countries (Van Lissa et al. 2022, 2–8; Taye et al. 2023). These findings support the inclusion of demographic, health, attitudinal and behavioural variables alongside the policy-period indicator.

Mask mandates provide a policy boundary for examining whether predictive relationships changed over time. An Australian study reported different leading predictors of mask wearing and broader protective behaviour before and after mandate onset (Ryan et al. 2025, 2–4, 8–11). Separate models were therefore fitted for the two policy periods.

Cross-national comparison requires a common analytical procedure because countries differ in demographic composition, survey timing and policy history. The same outcome definitions, preprocessing rules and modelling procedure were therefore applied to every retained country.

The individual-level data came from the Imperial College London/YouGov COVID-19 Behaviour Tracker Data Hub (Imperial College London and YouGov 2022). The anonymised survey files contain protective behaviours, demographic and household characteristics, employment, health conditions, symptoms, mental-health items, confidence in institutions, perceived risk and attitudes toward public-health measures. The data is a repeated cross-section: each row is a response for a specific date and location, and the same person will not be tracked in multiple rounds of investigation.

Policy timing came from the Oxford COVID-19 Government Response Tracker (OxCGRT) (Oxford COVID-19 Government Response Tracker 2023). The variable `H6M_Facial Coverings` records

face-covering policy strength by date, with subnational records identified through `RegionName` and `RegionCode` where available. Figure 1 summarises the linkage between survey responses and policy records.

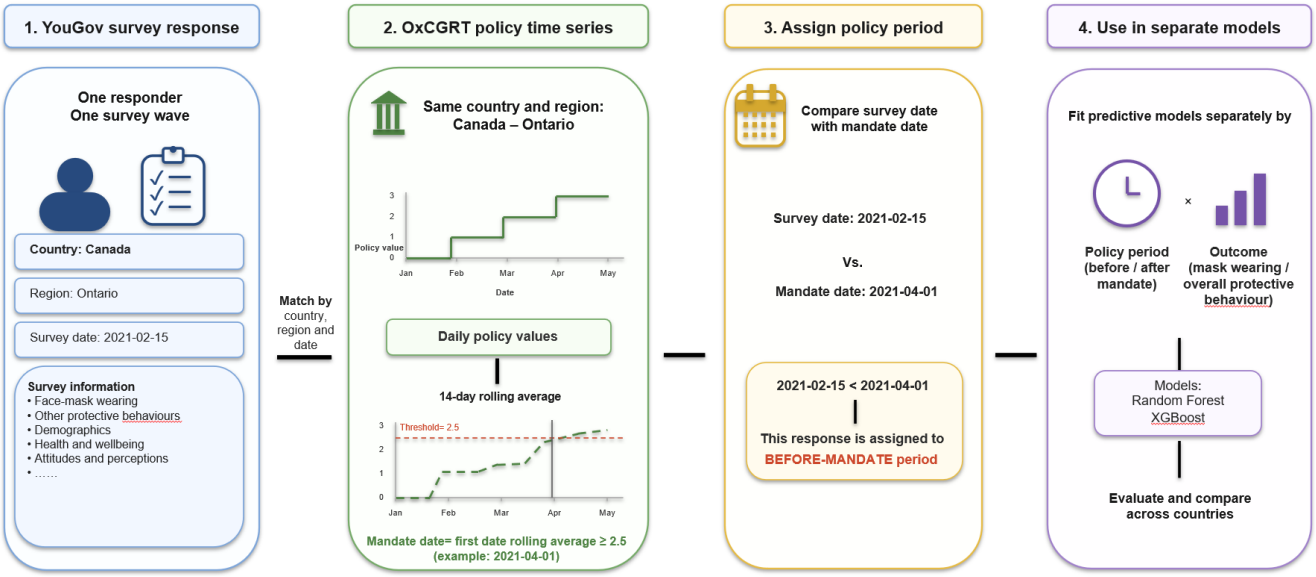

Figure 1: Linking survey responses to policy-period information.

The initial audit covered Australia, Brazil, Canada, China, India, the United Kingdom and the United States. China was not retained because its survey file lacked several columns required by the common workflow. The modelling analysis used the remaining six countries.

Random Forest and XGBoost were selected because decision-tree models can capture nonlinear relationships and interactions by repeatedly splitting observations according to predictor values. Random Forest combines predictions from many decision trees fitted to different samples and subsets of predictors (Breiman 2001, 5–8). XGBoost builds trees sequentially, with each new tree correcting errors made by the existing model, while regularisation penalises unnecessary model complexity to reduce overfitting (Chen and Guestrin 2016, 785–88). Both models have performed well in related research on COVID-19 protective behaviour in Australia (Ryan et al. 2025, 9).

ROC AUC was used to measure how well each model separated compliant from non-compliant responses across possible classification thresholds (Fawcett 2006, 868). Let $\hat{p}_i^+$ denote the predicted probability for a positive-class observation and $\hat{p}_j^-$ the predicted probability for a negative-class observation. ROC AUC can be interpreted as

$$\text{AUC} = \Pr\left(\hat{p}_i^+ > \hat{p}_j^-\right), \tag{1}$$

Higher AUC values indicate stronger discrimination between compliant and non-compliant responses.

The analysis compares predictors of face-mask wearing and overall protective behaviour before and after sustained mandate onset in six countries. It focuses on predictors that recur across countries and those that vary by policy period or national setting.

5 Methods

The analysis comprised data preparation, outcome construction, policy-period assignment, feature encoding, model fitting and evaluation. The complete code and supporting data files are available in the project GitHub repository (Zhao 2026). Table 2 links each stage to its implementation file.

Method stage	Implementation file
Column audit, file reading, country inclusion and initial renaming	`001 Uniform column format.ipynb`
Subnational region-name harmonisation	`001_2 State mapping.ipynb`
Missing-data screening, response recoding, outcome construction and mandate-period assignment	`002 Data Clean.ipynb`
Readable names for encoded variables	`003 Column Names update.ipynb`
Country-period-outcome train/test split	`004 Split.ipynb`
Random Forest fitting, tuning, AUC extraction and feature-importance extraction	`005 Random Forest (Model).ipynb`
XGBoost fitting, tuning, AUC extraction and feature-importance extraction	`006 XGBoost (Model).ipynb`

Table 2: Code files implementing the analysis workflow.

5.1 Data preparation workflow

The raw files were audited using a sample of 5,000 rows to check readability and column availability. The following encodings were tried in order:

- `utf-8-sig`
- `utf-8`
- `cp1252`
- `latin1`

Blank strings, single-space values and `__NA__` were treated as missing during reading. (`001 Uniform column format.ipynb, Cell 3`)

Required column names were first standardised. In the Brazil OxCGRT file, `H6_Facial Coverings` was renamed to `H6M_Facial Coverings`. In the YouGov files for Brazil, Canada and the United Kingdom, `region` was renamed to `state`. China was excluded because several required employment and policy-attitude variables were unavailable. Appendix D lists the variables used in the analysis and their meanings.

Subnational names were then harmonised so that YouGov survey locations could be matched to OxCGRT regions. This step corrected bilingual or alternative region names in Canada, mapped English regions in the United Kingdom to the corresponding OxCGRT region, standardised selected Indian territory names, and mapped District of Columbia to Washington DC for the United States. Accents were removed from region names after mapping. The Ladakh region was removed from the India OxCGRT file because it could not be matched to the YouGov region structure used in

the analysis. A final check was applied to verify that all YouGov regions used for modelling had corresponding OxCGRT region names. (`001_2 State mapping.ipynb, Cells 3-5`)

For each country, columns with more than 30% missing values were removed. A project-specific threshold of 30% was used to limit missingness while retaining a broad predictor set across countries. The variables `cantril_ladder`, `qweek`, `weight` and `household_children` were also excluded because they introduced additional missingness or non-comparable information. (`002 Data Clean.ipynb, Cell 9`)

5.2 Variable recoding and outcome construction

Variables with numerical meaning were recoded before row filtering:

- Household size values from 1 to 7 were retained, while 8 or more was coded as 8. Non-informative responses, including `Prefer not to say`, `Prefer not to answer`, `Don't know` and `N/A`, were treated as missing.
- Agreement-scale variables, including perceived COVID-19 severity and infection risk, were converted to ordered scores from 1 to 7.
- Protective-behaviour frequency items were recoded according to Table 3.
- Scores of 4 or 5 were classified as compliant, representing frequent or consistent performance.

(`002 Data Clean.ipynb`, Cell 5)

Original response	Recoded value
Not at all	1
Rarely	2
Sometimes	3
Frequently	4
Always	5

Table 3: Recoding of protective-behaviour frequency

Two binary outcomes were constructed:

- Face Mask Wearing.
- Overall Protective Behaviour.

Table 4 gives the full outcome definitions. (`002 Data Clean.ipynb`, Cell 10)

Outcome variable	Interpretation
Face Mask Wearing	Whether the respondent reported frequent or consistent mask wearing in the common mask-wearing survey item
Overall Protective Behaviour	Whether the respondent generally reported frequent or consistent engagement across the retained set of protective behaviours, such as distancing, hygiene-related behaviour, and related self-protective actions

Table 4: Definitions of the two binary outcome variables.

Let s_{ij} denote the recoded frequency score for respondent i on protective-behaviour item j. Let C denote the set of common protective-behaviour items retained across the six analysed countries, and let m denote the common face-mask item `i12_health_1`. The face-mask wearing score and binary outcome were defined as

$$S_i^{\text{mask}} = s_{im}, \tag{2}$$

$$Y_i^{\text{mask}} = \mathbb{I}\left(S_i^{\text{mask}} \geq 4\right), \tag{3}$$

where $Y_i^{\text{mask}} = 1$ indicates frequent or consistent self-reported mask wearing and $Y_i^{\text{mask}} = 0$ otherwise.

The overall protective behaviour score was calculated as the median of the common protective-behaviour items:

$$S_i^{\text{overall}} = \text{median}\{s_{ij} : j \in C\}, \tag{4}$$

$$Y_i^{\text{overall}} = \mathbb{I}\left(S_i^{\text{overall}} \geq 4\right). \tag{5}$$

The median summarised the respondent's typical frequency across all common protective-behaviour items, with complete responses required for the score to be calculated. A non-mask protective behaviour score was also calculated as the median over $C \setminus \{m\}$. This derived score was retained only as a predictor for the face-mask task and was removed from the overall protective behaviour task to avoid target leakage. (`002 Data Clean.ipynb, Cell 7`)

After the outcome variables had been created, rows with missing values in numeric or date variables were removed. Remaining missing values in categorical variables were retained as an explicit `N/A` category. This retained responses with missing categorical values. The original comorbidity indicators `d1_health_*` were collapsed into a single comorbidity variable. Respondents were classified as reporting comorbidity if any disease-specific comorbidity item was marked `Yes`. The values `No`, `Prefer not to say` and `N/A` were retained as distinct categories before dummy encoding. A relative survey-time variable was then created by grouping each respondent's survey completion date into 14-day intervals starting from the first retained date in that country. (`002 Data Clean.ipynb, Cells 4,8,11`)

5.3 Policy-period construction

The policy-period variable was derived from the OxCGRT `H6M_Facial Coverings` indicator. Policy dates were converted from `YYYYMMDD` format into calendar dates, and the face-covering indicator

was converted to a numeric value. For each subnational region with OxCGRT region records, a 14-day rolling mean of the face-covering indicator was calculated:

$$\bar{H}_{r,t}^{(14)} = \frac{1}{14} \sum_{k=0}^{13} H_{r,t-k}, \tag{6}$$

where $H_{r,t}$ is the face-covering indicator for region r on day t. Rolling means were calculated only when all 14 daily values were available. (`002 Data Clean.ipynb, Cells 3,6`)

The mandate start date for a region was defined using the threshold

$$\bar{H}_{r,t}^{(14)} \geq 2.5. \tag{7}$$

Where this threshold was reached, the first threshold-crossing date was used as the mandate start date. Where this threshold was not reached, the first date on which the region reached its maximum 14-day rolling mean was used. The region-specific mandate start dates produced by this rule are shown in Appendix A.2. The threshold and maximum-rolling-average rule is illustrated in Appendix A. (`002 Data Clean.ipynb, Cell 6`)

Each survey response was then assigned a binary mandate-period indicator:

$$P_i = \begin{cases} 0, & \text{if the survey date was before the assigned mandate start date,} \\ 1, & \text{if the survey date was on or after the assigned mandate start date.} \end{cases} \tag{8}$$

The value $P_i = 0$ defines the before-mandate period, and $P_i = 1$ defines the after-mandate period. This period variable was used to split each country into separate before-mandate and after-mandate modelling datasets. (`002 Data Clean.ipynb, Cells 6`)

5.4 Feature encoding and train-test split

Categorical predictors were converted using one-hot encoding, which represents each category as a binary 0/1 column; the first category was omitted as the reference. Binary outcomes were excluded from encoding. All variable recodings, name changes and readable labels are listed in Appendix D. (`002 Data Clean.ipynb, Cells 5,11`; `003 Column Names update.ipynb, Cells 3,4`)

For each retained country, four prediction tasks were created:

1. **Task 1:** before-mandate face-mask wearing;
2. **Task 2:** after-mandate face-mask wearing;
3. **Task 3:** before-mandate overall protective behaviour;
4. **Task 4:** after-mandate overall protective behaviour.

All predictors were checked to be numeric, and Boolean columns were converted from `TRUE/FALSE` to 1/0. (`004 Split.ipynb, Cells 5,6`)

The following variables were excluded before modelling:

- Respondent identifiers, survey dates and the mandate-period indicator, because they were used only for data organisation or period assignment.

- Binary outcomes and their corresponding continuous scores, to prevent target leakage. The non-mask protective-behaviour score was removed from Tasks 3 and 4 because its component items contributed to the overall protective-behaviour outcome.

The non-mask score was retained in Tasks 1 and 2 because it excluded the mask item. (`004 Split.ipynb, Cells 3,6`)

Each task used an 80/20 train-test split with random seed 1948883. Stratification was applied when both outcome classes contained at least two observations; otherwise, an unstratified random split would be used. This fallback was not required because all 24 tasks satisfied the condition. Model fitting and tuning used only the training data, while the held-out test data were reserved for final evaluation. (`004 Split.ipynb, Cells 3,5,6`)

5.5 Model training

Two ensemble tree models were fitted for each country, policy period and outcome task: Random Forest and XGBoost. This produced a separate fitted model for each combination of country, period and outcome. The positive-class proportion varied by country, outcome and policy period. To reduce the effect of class imbalance, the minority class was randomly oversampled during training. Oversampling was applied only to the training portion of each cross-validation split and not to the validation fold or final held-out test set. (`005 Random Forest (Model).ipynb`, Cell 4; `006 XGBoost (Model).ipynb`, Cell 4)

Hyperparameters were tuned using a randomised search. For each model and task, 40 hyperparameter combinations were sampled using random seed 1948883. The tuning metric was mean cross-validated ROC AUC. Cross-validation was performed within the training set using shuffled stratified folds with the same random seed. The maximum number of folds was five, but fewer folds were used when the minority class contained fewer than five training observations. A task would be skipped if the training data contained fewer than two minority-class observations, although this did not occur. After the best hyperparameter combination was selected, the model was refitted to the full training set. (`005 Random Forest (Model).ipynb`, Cells 3,4,5; `006 XGBoost (Model).ipynb`, Cells 3,4,5)

5.5.1 Random Forest implementation

Each Random Forest model used 250 trees and the common random seed 1948883. Parallel processing was used to reduce computation time. The tuned hyperparameters are shown in Table 5. (`005 Random Forest (Model).ipynb, Cell 4`)

Hyperparameter	Values
Depth	5, 8, 10, 12, 15, 20, unrestricted
Minimum samples for split	2, 5, 10, 20, 30
Minimum samples per leaf	1, 2, 5, 10
Feature subset	sqrt, log2, 0.3, 0.5, 0.7, all
Bootstrap	true

Table 5: Random Forest hyperparameter search space.

5.5.2 XGBoost implementation

Each XGBoost model used 250 boosting trees, a binary classification objective, log-loss for model evaluation, histogram-based tree construction and the common random seed 1948883. Parallel processing was used to reduce computation time. The tuned hyperparameters are shown in Table 6. (`006 XGBoost (Model).ipynb, Cell 4`)

Hyperparameter	Values
Depth	3, 4, 5, 6, 8, 10
Learning rate	0.01, 0.03, 0.05, 0.10, 0.20
Row sample	0.6, 0.7, 0.8, 0.9, 1.0
Column sample	0.5, 0.6, 0.7, 0.8, 0.9, 1.0
Child weight	1, 3, 5, 7
Gamma	0, 0.01, 0.1, 0.3, 0.5, 1.0
L_1 penalty	0, 0.01, 0.1, 1.0
L_2 penalty	0.5, 1.0, 2.0, 5.0

Table 6: XGBoost hyperparameter search space.

5.6 Evaluation and feature-importance extraction

Cross-validation ROC AUC was used only to select hyperparameters within the 80% training set. After the best settings were selected, each model was refitted using the complete training set and evaluated once on the untouched 20% test set. Test ROC AUC was calculated from the predicted probability of the positive class. Every test set contained both outcome classes, so an AUC value was obtained for all 48 fitted models. (`005 Random Forest (Model).ipynb`, Cell 4; `006 XGBoost (Model).ipynb`, Cell 4)

Feature importance was extracted from each final refitted model, using impurity-based importance for Random Forest and the built-in tree-based importance for XGBoost. A complete importance table was saved for every model. For each country and outcome, the before- and after-mandate values were combined, state indicators and explicit `N/A` categories were excluded, and features were ranked by their larger importance across the two periods. The ten highest-ranked features were displayed and grouped as demographics, self-protective behaviours, perception of illness threat, health and wellbeing, time, or other. (`005 Random Forest (Model).ipynb`, Cells 4, 5 and 7; `006 XGBoost (Model).ipynb`, Cells 4,5,7)

6 Results

6.1 Data preparation workflow

The initial audit covered 14 source files. Thirteen were read using `utf-8-sig`, while the United Kingdom YouGov file required `cp1252` before being exported in the common `utf-8-sig` format. The Brazil policy column was renamed to `H6M_Facial Coverings`, and `region` was renamed to `state` for Brazil, Canada and the United Kingdom. China was excluded because several required variables were unavailable, leaving six countries in the final analysis.

The initial location-matching check found no unmatched regional names for Australia or Brazil. Mismatches were found for Canada, India, the United Kingdom and the United States. Table 7 lists the corrections; the final check found no unmatched YouGov locations.

Country	Number of failures	Harmonisation result
Australia	0	No regional-name change required
Brazil	0	No regional-name change required
Canada	7	Bilingual or accented province and territory labels were mapped to the corresponding OxCGRT names
United Kingdom	9	The nine English survey regions were mapped to the OxCGRT region `England`
India	4	Four alternative territory/state labels were standardised; the unmatched OxCGRT region `Ladakh` was removed
United States	1	`District of Columbia` was mapped to `Washington DC`

Table 7: Data preprocessing Result

6.2 Variable recoding and outcome construction

Fourteen protective-behaviour items remained available in all six countries after missingness screening. These items were used to construct the variables in Table 8, and their exact composition is listed in Appendix E.

Constructed variable	Number of common items	Role in the modelling workflow
Face Mask Wearing	1	Formal prediction outcome based on `i12_health_1`
Non-Mask Protective Behavior	13	Auxiliary non-mask behaviour measure; the score was retained only for face-mask prediction
Overall Protective Behavior	14	Formal prediction outcome based on the median of all common protective-behaviour items

Table 8: Behaviour variables

After recoding, 156,772 of 227,130 survey records remained, an overall retention rate of 69.0%. Figure 2 shows the country-level totals.

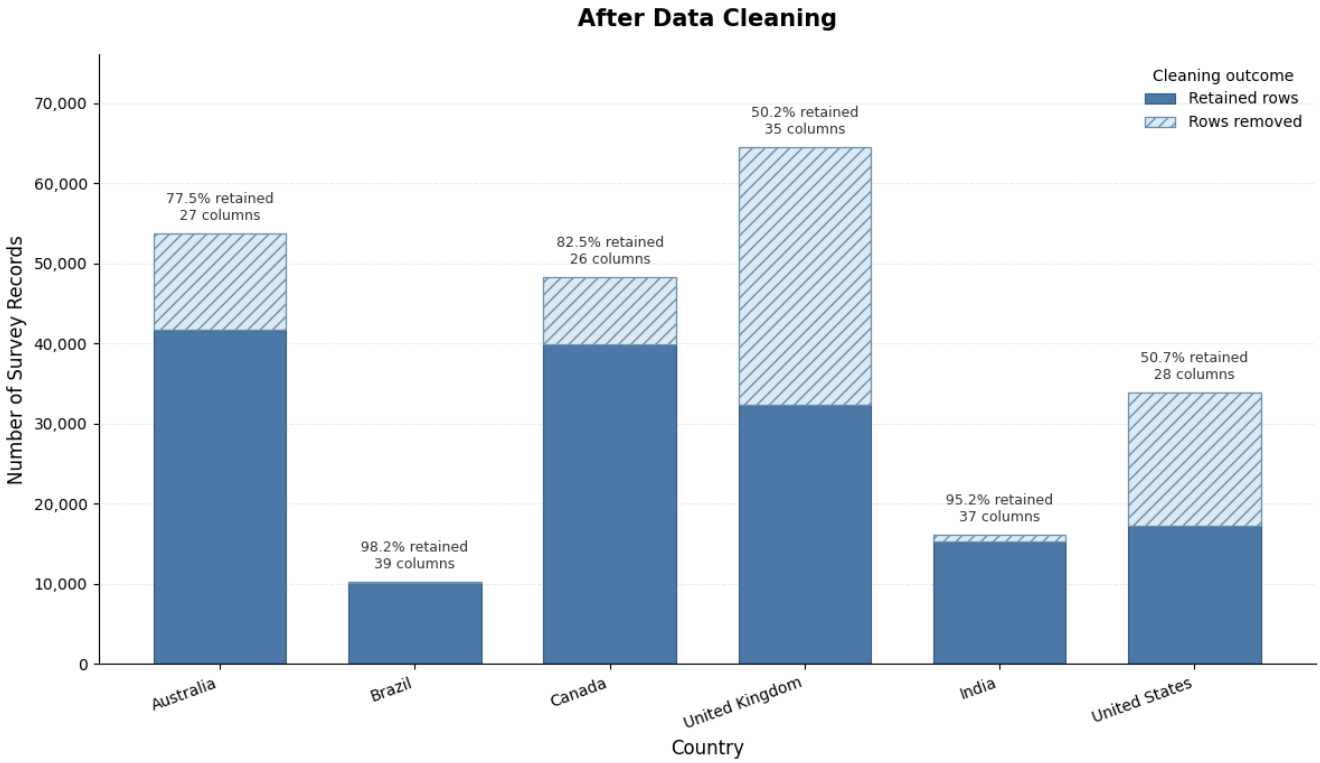

Figure 2: Result After Data Cleaning

The recoding stage also produced one collapsed comorbidity variable, one relative 14-day survey-time variable, and explicit `N/A` levels for remaining categorical missing values. The original `i12_health_*` items were removed after the behaviour variables had been constructed.

6.3 Policy-period construction

The regional policy audit produced 138 region-specific mandate start dates across the six retained countries. Every cleaned survey response was assigned to either the before-mandate or after-mandate period; no records remained with an unknown mandate-period value. Figure 3 shows the resulting country-level distribution.

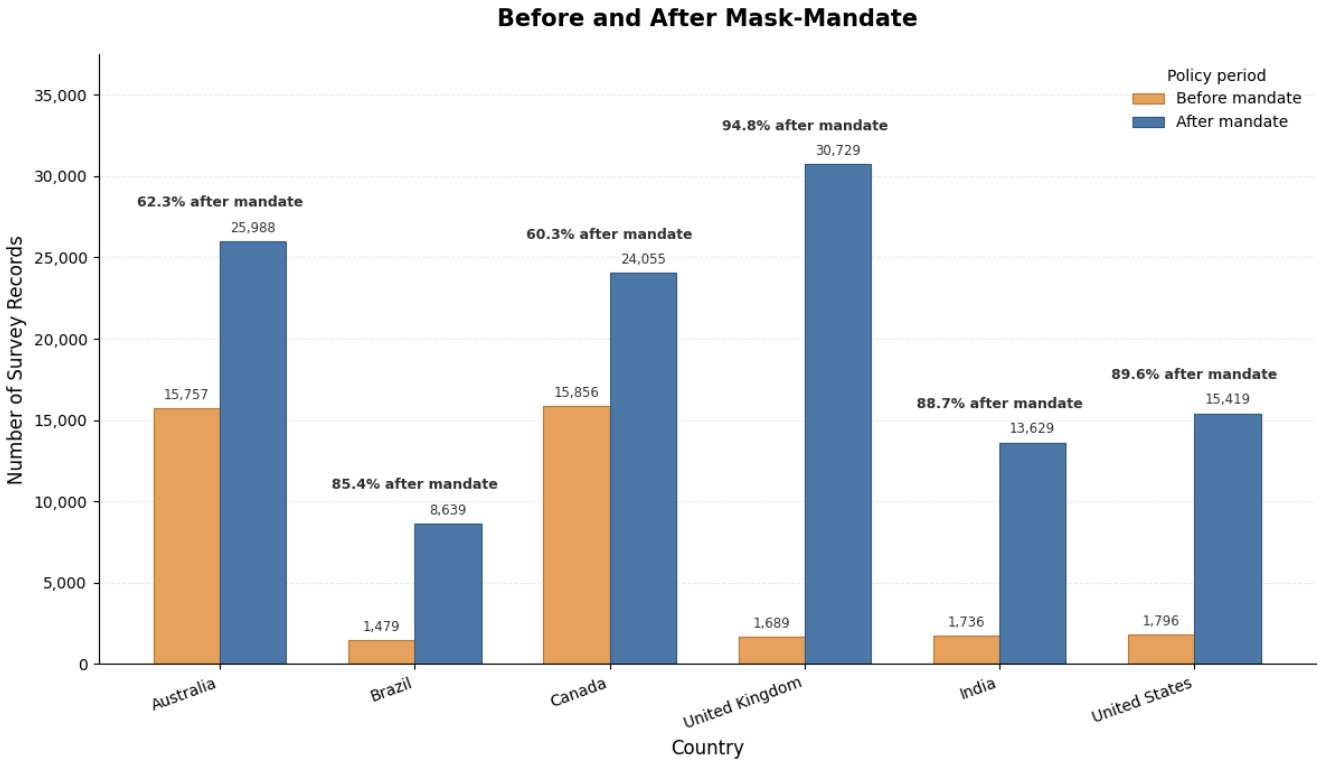

Figure 3: Result of before-mandate and after-mandate periods

The after-mandate share was 60.3% in Canada and 62.3% in Australia. It exceeded 85% in Brazil, India, the United Kingdom and the United States.

6.4 Feature encoding and train-test split

The number of encoded columns differed across countries. After removing identifiers, dates, policy-period fields and outcome-related leakage variables, the face-mask task contained one additional predictor relative to the overall protective-behaviour task: the non-mask protective behaviour score. The resulting dimensions are shown in Table 9.

Country	Encoded columns	Face-mask predictors	Overall-behaviour predictors
Australia	67	59	58
Brazil	130	122	121
Canada	71	63	62
United Kingdom	81	73	72
India	127	119	118
United States	102	94	93

Table 9: Result After encoding

All 4 modelling tasks were successfully constructed for each of the 6 countries by combining the two policy periods with the two prediction outcomes. This produced 24 country–period–outcome tasks and, because each task generated separate training and test files for the predictors and target, 96 split files in total. The planned 80/20 training–test allocation was applied consistently across all tasks using the common random seed specified in the Methods. Stratification was performed with respect to the binary target within each task, thereby preserving the observed outcome-class proportions as closely as possible between the training and test sets. The resulting feature matrices contained only numerical predictors, including binary dummy variables represented as 0 or 1, and all target vectors retained valid binary outcome classes. Consequently, a complete and internally consistent set of 24 modelling datasets was available for subsequent model fitting and evaluation.

6.5 Model training

All 24 country–period–outcome tasks were successfully retained for modelling, and both Random Forest and XGBoost produced a fitted model for every task. This resulted in 48 final models across the six countries. No task was excluded because each training set contained sufficient observations in both outcome classes for stratified cross-validation. The randomised search evaluated 960 candidate configurations for Random Forest and 960 for XGBoost. A final model was selected and refitted for each task, allowing held-out test ROC AUC to be calculated for all 48 models.

The resulting test ROC AUC values are reported in Table 10. For each country–period–outcome task, the larger of the two AUC values is shown in bold to indicate which model achieved the stronger held-out discrimination.

Country	Outcome	RF	XGBoost	Country	Outcome	RF	XGBoost
Australia	Face mask wearing Before mandate	0.844	**0.854**	UK	Face mask wearing Before mandate	0.687	**0.695**
	Face mask wearing After mandate	0.909	**0.912**		Face mask wearing After mandate	0.715	**0.724**
	Protective behaviour Before mandate	0.767	**0.772**		Protective behaviour Before mandate	0.797	**0.799**
	Protective behaviour After mandate	0.826	**0.831**		Protective behaviour After mandate	0.829	**0.838**
Brazil	Face mask wearing Before mandate	0.783	**0.785**	India	Face mask wearing Before mandate	0.757	**0.764**
	Face mask wearing After mandate	0.652	**0.653**		Face mask wearing After mandate	**0.856**	0.851
	Protective behaviour Before mandate	**0.712**	0.705		Protective behaviour Before mandate	**0.823**	0.801
	Protective behaviour After mandate	**0.829**	0.827		Protective behaviour After mandate	0.827	**0.831**
Canada	Face mask wearing Before mandate	**0.842**	0.840	US	Face mask wearing Before mandate	0.842	**0.845**
	Face mask wearing After mandate	0.842	**0.845**		Face mask wearing After mandate	0.856	**0.860**
	Protective behaviour Before mandate	0.847	**0.851**		Protective behaviour Before mandate	0.840	**0.847**
	Protective behaviour After mandate	0.831	**0.837**		Protective behaviour After mandate	0.850	**0.858**

Table 10: Model comparison (AUC)

XGBoost had the higher test AUC in 19 of the 24 tasks, while Random Forest had the higher value in five tasks. No task was tied. The mean test AUC was 0.809 for XGBoost and 0.807 for Random Forest.

6.6 XGBoost feature importance

XGBoost was used for the final feature-importance figures because it had the higher held-out test AUC in 19 of the 24 tasks. Each figure shows the ten features with the largest importance in either policy period after state indicators and explicit `N/A` categories were removed. The left and right bars correspond to the before-mandate and after-mandate models, respectively; the upper and lower panels show face-mask wearing and overall protective behaviour.

6.6.1 Australia

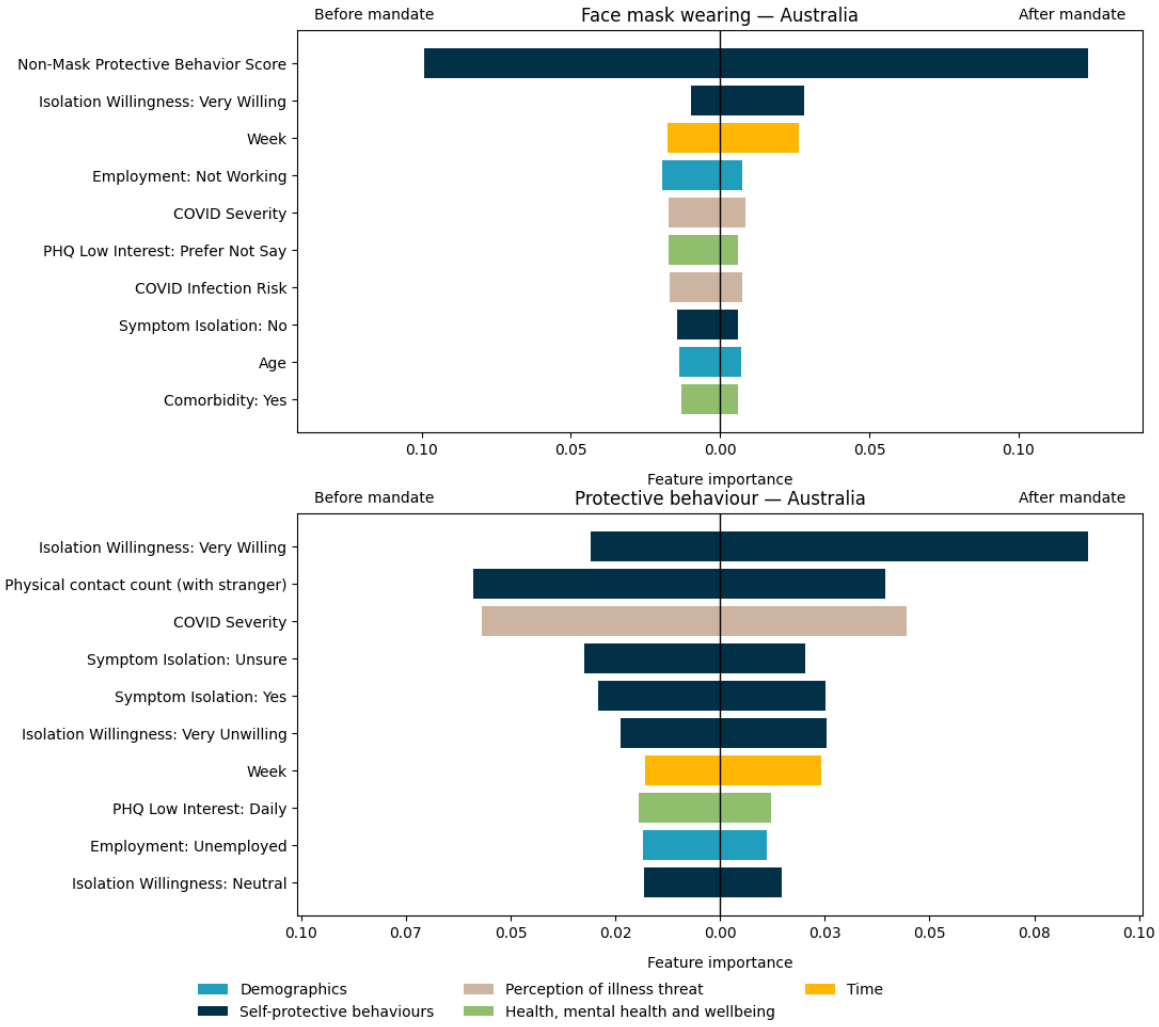

Figure 4: Feature importance (Australia)

In Figure 4, Non-Mask Protective Behavior Score ranked first for face-mask wearing before and after the mandate. Isolation Willingness: Very Willing ranked next, with a higher importance after the mandate. Week, employment status, COVID Severity, COVID Infection Risk, symptom-isolation responses, age and comorbidity also appeared among the ten displayed features. For overall protective behaviour, Physical contact count (with stranger) ranked first before the mandate, while Isolation Willingness: Very Willing ranked first after the mandate. COVID Severity, symptom-isolation responses, Week, PHQ Low Interest: Daily and employment status were also included among the displayed features.

6.6.2 Brazil

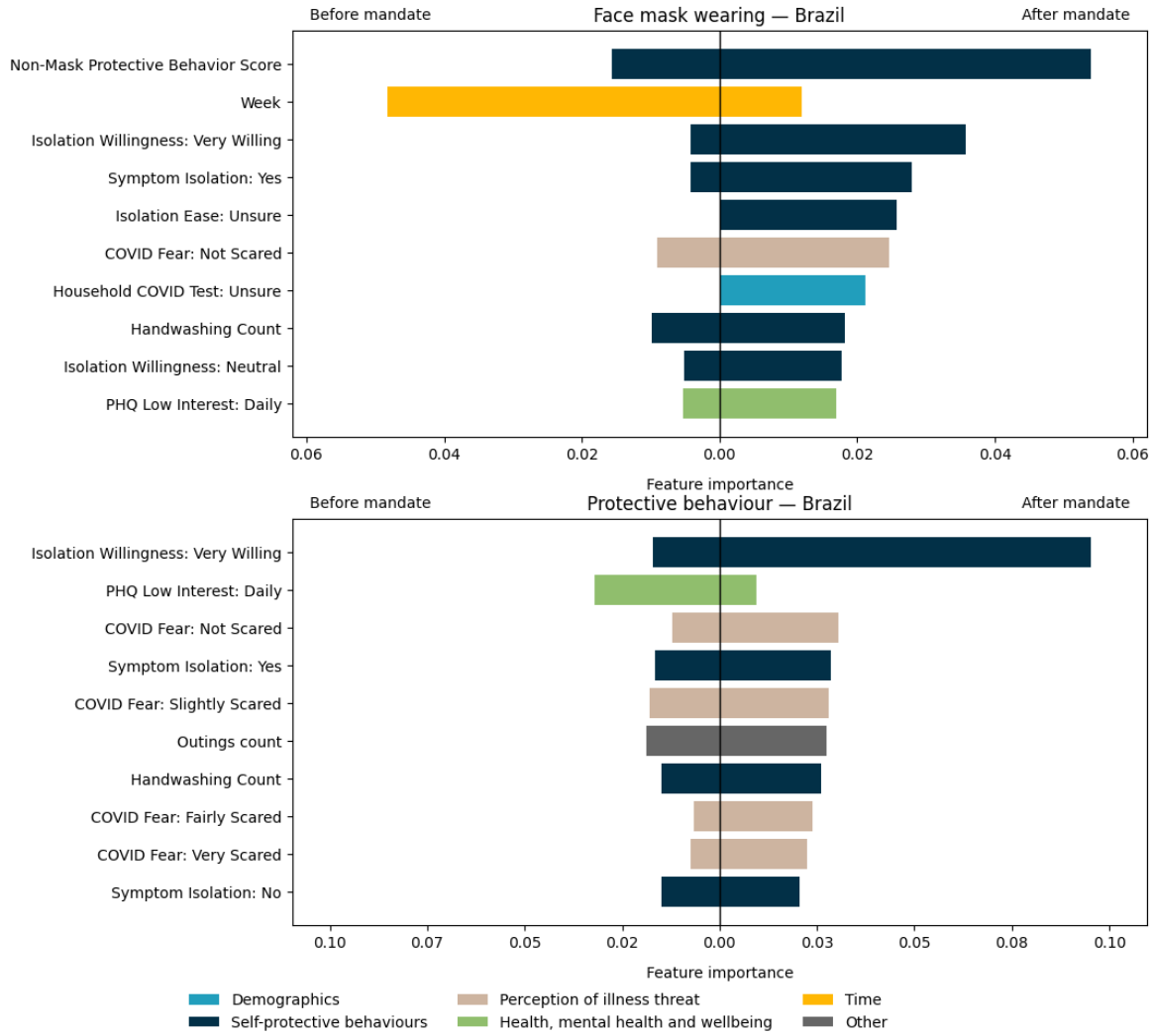

Figure 5: Feature importance (Brazil)

In Figure 5, Week ranked first for face-mask wearing before the mandate, while Non-Mask Protective Behavior Score ranked first after the mandate. Isolation Willingness: Very Willing, symptom-isolation responses, household COVID-testing status, handwashing count and PHQ Low Interest: Daily also appeared among the ten displayed features. For overall protective behaviour, PHQ Low Interest: Daily ranked first before the mandate, whereas Isolation Willingness: Very Willing ranked first after the mandate. COVID-fear categories, symptom-isolation responses, outings count and handwashing count were also included.

6.6.3 Canada

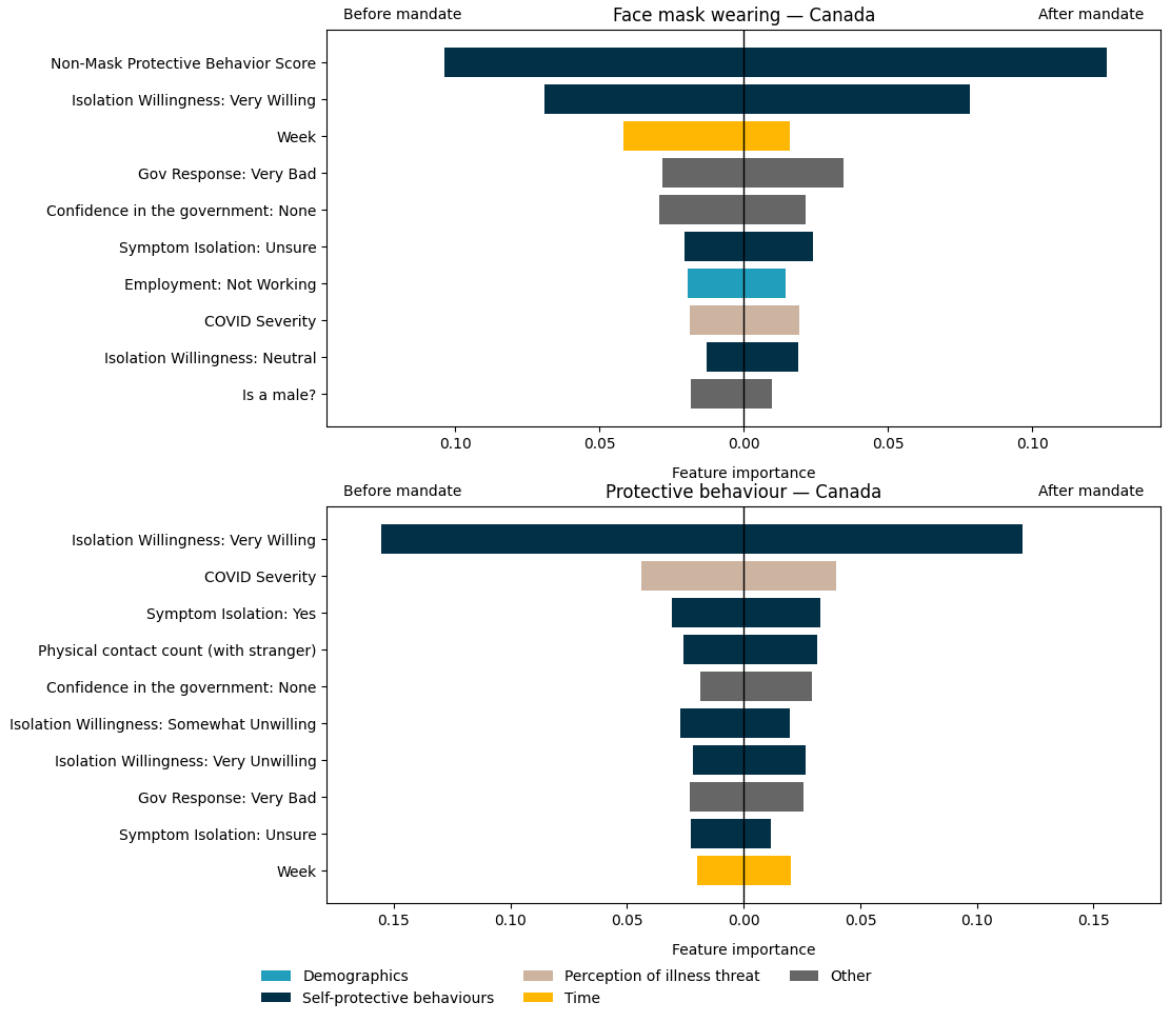

Figure 6: Feature importance (Canada)

In Figure 6, Non-Mask Protective Behavior Score ranked first for face-mask wearing in both policy periods, followed by Isolation Willingness: Very Willing. Week had a higher importance before the mandate, while confidence in government, perceived government response, employment status, COVID Severity and symptom-isolation responses also appeared among the ten displayed features. For overall protective behaviour, Isolation Willingness: Very Willing ranked first before and after the mandate. COVID Severity, physical contact count (with stranger), confidence in government and additional isolation-related variables were also included.

6.6.4 India

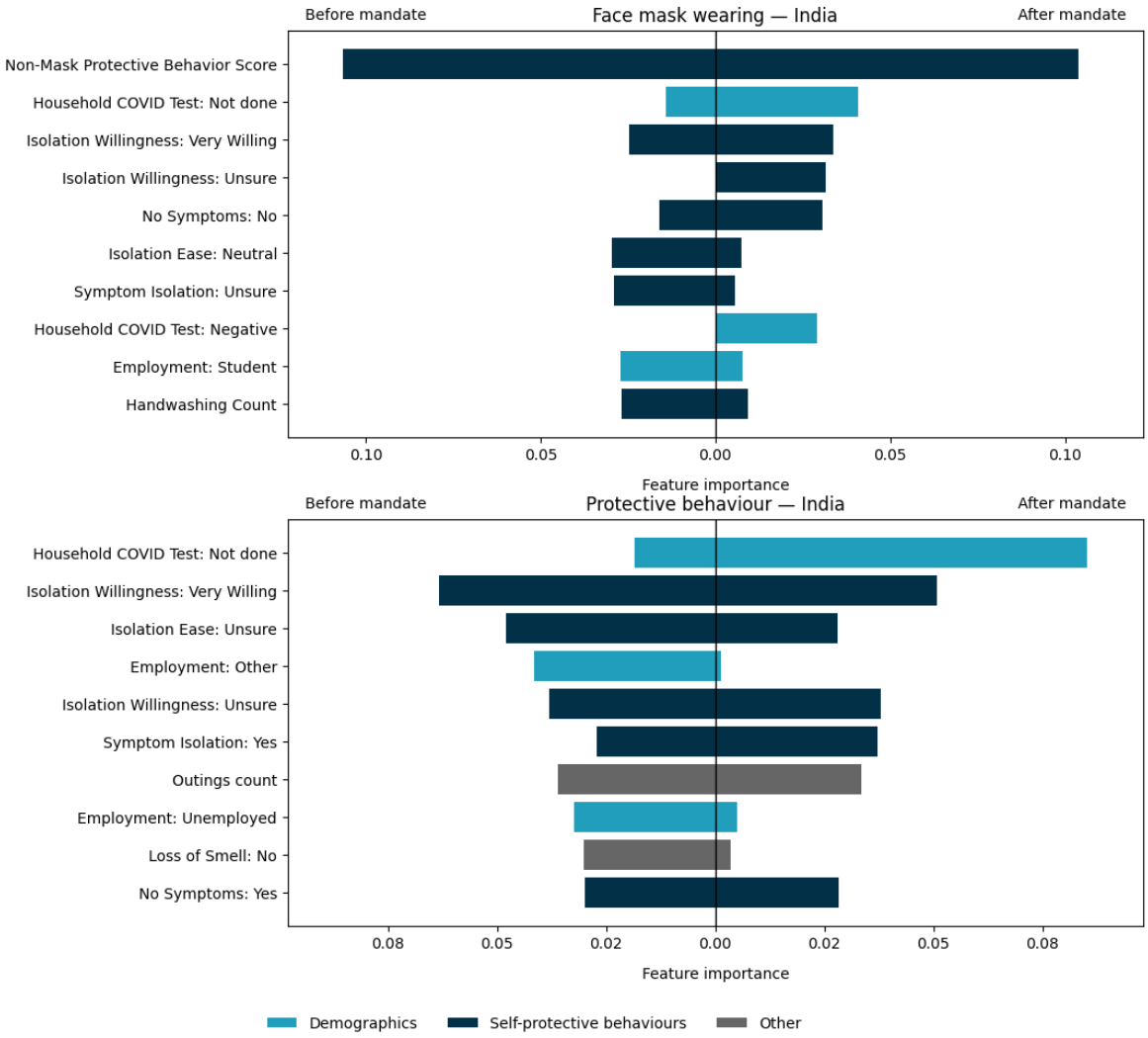

Figure 7: Feature importance (India)

In Figure 7, Non-Mask Protective Behavior Score ranked first for face-mask wearing before and after the mandate. Household COVID Test: Not done, isolation-willingness categories, symptom-isolation responses, employment status and handwashing count also appeared among the ten displayed features. For overall protective behaviour, Isolation Willingness: Very Willing ranked first before the mandate, while Household COVID Test: Not done ranked first after the mandate. Employment categories, symptom-isolation responses, outings count and health-related variables were also included.

6.6.5 United Kingdom

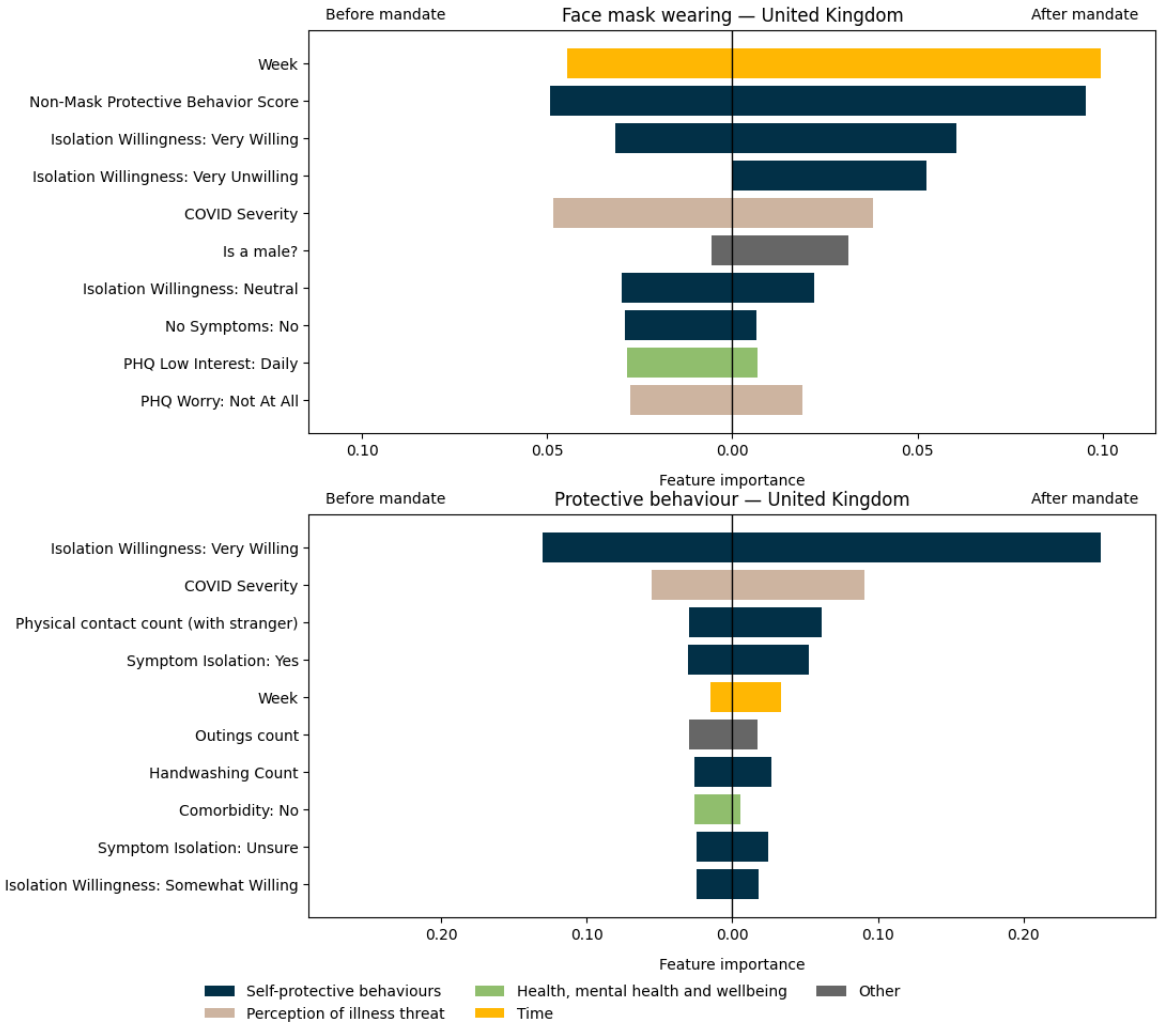

Figure 8: Feature importance (United Kingdom)

In Figure 8, Non-Mask Protective Behavior Score ranked first for face-mask wearing before the mandate, while Week ranked first after the mandate. Isolation Willingness: Very Willing, COVID Severity, other isolation-willingness categories and mental-health variables also appeared among the ten displayed features. For overall protective behaviour, Isolation Willingness: Very Willing ranked first in both policy periods, followed by COVID Severity. Physical contact count (with stranger), symptom isolation, Week, outings count, handwashing count and comorbidity were also included.

6.6.6 United States

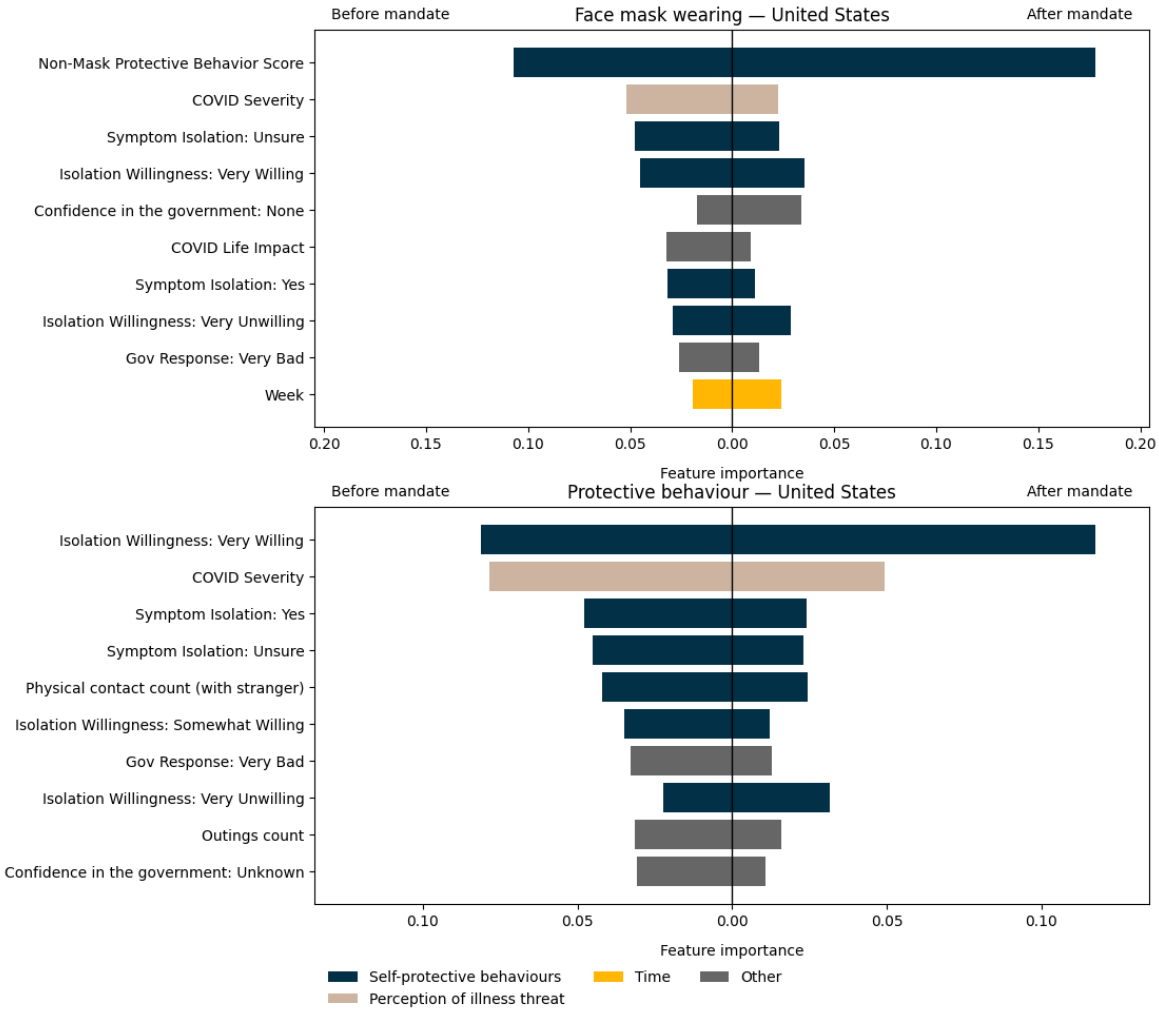

Figure 9: Feature importance (United States)

In Figure 9, Non-Mask Protective Behavior Score ranked first for face-mask wearing before and after the mandate. COVID Severity, symptom-isolation responses, Isolation Willingness: Very Willing, employment status, confidence in government, perceived government response and Week also appeared among the ten displayed features. For overall protective behaviour, Isolation Willingness: Very Willing ranked first in both policy periods, followed by COVID Severity. Symptom isolation, physical contact count (with stranger), additional isolation-willingness categories, outings count and confidence in government were also included.

7 Discussion and Conclusion

7.1 Discussion

The models achieved moderate to strong discrimination across most country, policy-period and outcome combinations, indicating that the survey variables contained meaningful predictive information about protective behaviour. XGBoost achieved the higher test ROC AUC in 19 of the 24 tasks, although its average advantage over Random Forest was small. The feature-importance results should therefore be interpreted as describing the predictive structure identified by a generally well-performing tree-based model rather than demonstrating a substantial superiority of one algorithm. Performance also varied across tasks: discrimination was strongest for face-mask wearing after mandate onset in Australia and weakest for the corresponding task in Brazil. Feature rankings from lower-performing models should consequently be interpreted with greater caution.

The predictor sets were not identical across countries because missingness screening and differences in survey content left each country with a shared core of variables and a set of additional country-specific predictors. The two outcomes and their construction were nevertheless consistent across all six countries. Cross-country comparisons should therefore focus on recurring predictor types and broader ranking patterns rather than assume that every model selected from exactly the same candidate variables. The absence of a feature from a country's leading predictors may reflect limited predictive value, but it may also indicate that the variable was unavailable or represented differently in that country.

For face-mask wearing, the clearest recurring result was the importance of the Non-Mask Protective Behavior Score. It was the leading predictor in both policy periods in Australia, Canada, India and the United States, before the mandate in the United Kingdom, and after the mandate in Brazil. Respondents who frequently practised other preventive behaviours were therefore also more likely to report frequent mask wearing. Because the score excluded the mask item, this association was not produced by direct overlap with the outcome. Instead, it suggests that mask wearing formed part of a broader protective orientation rather than representing an isolated behaviour. Survey Week was the principal exception, becoming the leading mask-wearing predictor before the mandate in Brazil and after the mandate in the United Kingdom. Its importance indicates that mask behaviour changed over the course of the pandemic in ways not fully captured by the measured demographic and attitudinal predictors. Such temporal variation may reflect simultaneous changes in infection levels, public communication, social expectations and familiarity with mask use, although the present analysis cannot separate these processes.

For overall protective behaviour, Isolation Willingness: Very Willing was the most consistent leading predictor. It ranked first in both periods in Canada, the United Kingdom and the United States, after the mandate in Australia and Brazil, and before the mandate in India. Willingness to self-isolate may therefore represent a broader readiness to accept behavioural restrictions in response to infection risk. Perceived COVID-19 severity, symptom-related isolation, physical contact, outings and handwashing also appeared repeatedly, indicating that overall protective behaviour was associated with a combination of risk perception, willingness to restrict activity and existing behavioural practices.

Alongside these common patterns, the models identified clear national differences. Government confidence and evaluations of the government response were particularly visible in Canada and the United States. Household COVID-testing status and employment variables were more promi-

nent in India, while mental-health measures, COVID-related fear, handwashing and survey timing appeared more strongly in Brazil. Physical contact was the leading overall-behaviour predictor before the mandate in Australia, whereas Week became the leading face-mask predictor after the mandate in the United Kingdom. These findings suggest that the common behavioural predictors operated within different social, institutional and epidemiological settings. This pattern is consistent with previous studies showing that protective behaviour is related to risk perception, trust, psychological factors and national context (Roma et al. 2020, 2–6; Van Lissa et al. 2022, 2–8; Taye et al. 2023, 11–12). The cross-national results also extend the earlier Australian evidence of differences before and after mandate onset (Ryan et al. 2025, 2–4, 8–11), while showing that the form of these changes was not uniform across countries.

Predictor rankings changed between the pre- and post-mandate periods in every country, but the extent and structure of change were not uniform. Canada and the United States retained relatively stable leading predictors, and the dominant face-mask predictor also remained unchanged in Australia and India. More substantial changes occurred in other tasks. In Brazil, the leading face-mask predictor shifted from Week to the non-mask behaviour score, while the leading overall-behaviour predictor shifted from low interest to isolation willingness. In India, the leading overall-behaviour feature changed from isolation willingness to household COVID-testing status. In the United Kingdom, the leading face-mask feature changed from the non-mask behaviour score to Week. Australia also showed a change in the overall-behaviour model, from physical contact before the mandate to isolation willingness afterwards. Mandate onset was therefore associated with a reordering of predictive relationships, but not with a single transition shared equally by all countries.

These differences should not be interpreted as causal effects of mask mandates. The policy boundary separated observations into before- and after-mandate periods, but infection prevalence, public information, personal experience, social norms and the composition of survey responses also changed over time. Feature importance measures the contribution of a variable to prediction within a fitted model; it does not show that changing that variable would cause behaviour to change. The use of self-reported behaviour may also introduce recall or social-desirability bias, and the country-specific predictor sets limit direct comparison of exact importance values. The conclusions should therefore be understood as predictive associations rather than causal estimates.

7.2 Conclusion

The findings answer the three research questions as follows:

1. **Did the important predictors change within countries before and after mask mandates?**
   Predictor rankings changed between the two policy periods in every country, although some leading features remained stable. Face-mask wearing was most consistently predicted by engagement in other protective behaviours, while overall protective behaviour was most consistently predicted by willingness to self-isolate.

2. **Were these changes consistent across countries?**
   No single pattern of change was shared by all countries. Some countries retained similar leading predictors across the two periods, while others showed larger changes involving survey timing, testing status, employment, mental health or physical contact.

3. **Were there common cross-national predictors alongside country-specific predictors?**
   Both common and country-specific patterns were identified. Broader protective orientation, isolation willingness and perceived COVID-19 severity appeared across several countries, while their relative importance and the contribution of other predictors varied by country and policy period.

Overall, the results indicate shared predictive patterns alongside distinct country-level differences rather than one universal change following mask-mandate onset.

8 Acknowledgements

I would like to express my sincere gratitude to Anthony and Liangning for their guidance, feedback and support throughout this project. Their advice helped improve both the technical analysis and the presentation of the final report. I also acknowledge Imperial College London and YouGov for providing the COVID-19 Behaviour Tracker data, and the Oxford COVID-19 Government Response Tracker team for making the policy data publicly available.

I acknowledge the use of generative AI as a limited writing aid during the preparation of this report. AI were used primarily to improve sentence clarity, grammar and academic phrasing. All research decisions, data processing, model implementation, interpretation of results and final revisions were completed and verified by myself.

All the code and data for implementing these features can be found on my GitHub (Zhao 2026).

9 References

Breiman, Leo. 2001. "Random Forests." *Machine Learning* 45 (1): 5–32. https://doi.org/10.1023/A:1010933404324.

Chen, Tianqi, and Carlos Guestrin. 2016. "XGBoost: A Scalable Tree Boosting System." *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–94. https://doi.org/10.1145/2939672.2939785.

Fawcett, Tom. 2006. "An Introduction to ROC Analysis." *Pattern Recognition Letters* 27 (8): 861–74. https://doi.org/10.1016/j.patrec.2005.10.010.

Galasso, Vincenzo, Vincent Pons, Paola Profeta, Michael Becher, Sylvain Brouard, and Martial Foucault. 2020. "Gender Differences in COVID-19 Attitudes and Behavior: Panel Evidence from Eight Countries." *Proceedings of the National Academy of Sciences* 117 (44): 27285–91. https://doi.org/10.1073/pnas.2012520117.

Haischer, Michael H., Rachel Beilfuss, Meggie Rose Hart, et al. 2020. "Who Is Wearing a Mask? Gender-, Age-, and Location-Related Differences During the COVID-19 Pandemic." *PLOS ONE* 15 (10): e0240785. https://doi.org/10.1371/journal.pone.0240785.

Imperial College London, and YouGov. 2022. *Imperial College London–YouGov COVID-19 Behaviour Tracker Data Hub*. GitHub repository. https://github.com/YouGov-Data/covid-19-tracker.

Lisi, Matteo P., Marina Scattolin, Martina Fusaro, and Salvatore Maria Aglioti. 2021. "A Bayesian Approach to Reveal the Key Role of Mask Wearing in Modulating Projected Interpersonal Distance During the First COVID-19 Outbreak." *PLOS ONE* 16 (8): e0255598. https://doi.org/10.1371/journal.pone.0255598.

Oxford COVID-19 Government Response Tracker. 2023. *Oxford COVID-19 Government Response Tracker: Final Policy Dataset*. GitHub repository. https://github.com/OxCGRT/covid-policy-tracker.

Roma, Paolo, Merylin Monaro, Laura Muzi, et al. 2020. "How to Improve Compliance with Protective Health Measures During the COVID-19 Outbreak: Testing a Moderated Mediation Model and Machine Learning Algorithms." *International Journal of Environmental Research and Public Health* 17 (19): 7252. https://doi.org/10.3390/ijerph17197252.

Ryan, Matthew, Jinjing Ye, Justin Sexton, Roslyn I. Hickson, and Emily Brindal. 2025. "Face Mask Mandates Alter Major Determinants of Adherence to Protective Health Behaviours in Australia." *Royal Society Open Science* 12 (3): 241941. https://doi.org/10.1098/rsos.241941.

Taye, Alemayehu D., Liyousew G. Borga, Samuel Greiff, Claus Vögele, and Conchita D'Ambrosio. 2023. "A Machine Learning Approach to Predict Self-Protecting Behaviors During the Early Wave of the COVID-19 Pandemic." *Scientific Reports* 13 (1): 6121. https://doi.org/10.1038/

s41598-023-33033-1.

Van Lissa, Caspar J., Wolfgang Stroebe, Michelle R. vanDellen, et al. 2022. "Using Machine Learning to Identify Important Predictors of COVID-19 Infection Prevention Behaviors During the Early Phase of the Pandemic." *Patterns* 3 (4): 100482. https://doi.org/10.1016/j.patter.2022.100482.

Zhao, Zixuan. 2026. *COVID-19 Protective Behavior Cross-National Analysis*. GitHub repository. https://github.com/ZixaunZhao/COVID-19-Protective-Behavior-Cross-National-Analysis.

A Mask-policy trajectories and mandate-date rule

A.1 Country-level mask mandate dates

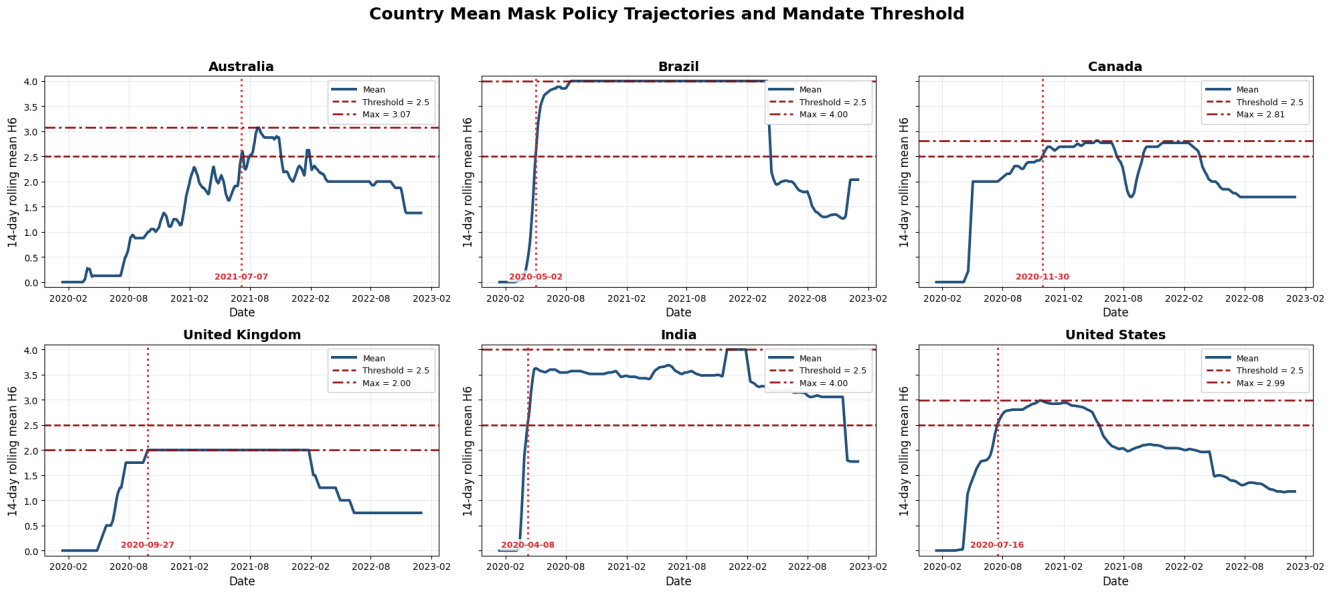

Figure A.1: Diagnostic mask-policy trajectories and mandate-date rule.

Figure A.1 shows the country-level mean of the 14-day rolling average of the OxCGRT `H6M_Facial Coverings` indicator. The horizontal dashed line marks the threshold of 2.5, the dash-dotted line marks the maximum displayed value, and the vertical dotted line marks the selected mandate date.

The threshold was applied to the 14-day rolling average rather than to the raw daily value so that mandate onset was based on a sustained policy level rather than a short one-day change in the policy data. Where a region-level rolling average reached or exceeded 2.5, the first threshold-crossing date was used. Where a region did not reach this threshold, the first date on which the region reached its maximum 14-day rolling average was used instead. After a mandate start date was identified, survey responses dated before that point were assigned to the before-mandate period, while responses dated on or after that point were assigned to the after-mandate period.

The figure is a diagnostic check rather than the basis of the region-level assignments. Because it averages across regions, a country-level trajectory may not cross the threshold in the same way as every regional series.

A.2 Regional mandate start dates

Figures A.2.1–A.2.6 show the region-specific mandate start dates used to assign the policy period in each of the six retained countries. Each point represents the selected mandate start date for one region, while the vertical dotted line shows the corresponding country-level date obtained from the rolling-mean rule described above.

A.2.1 Australia

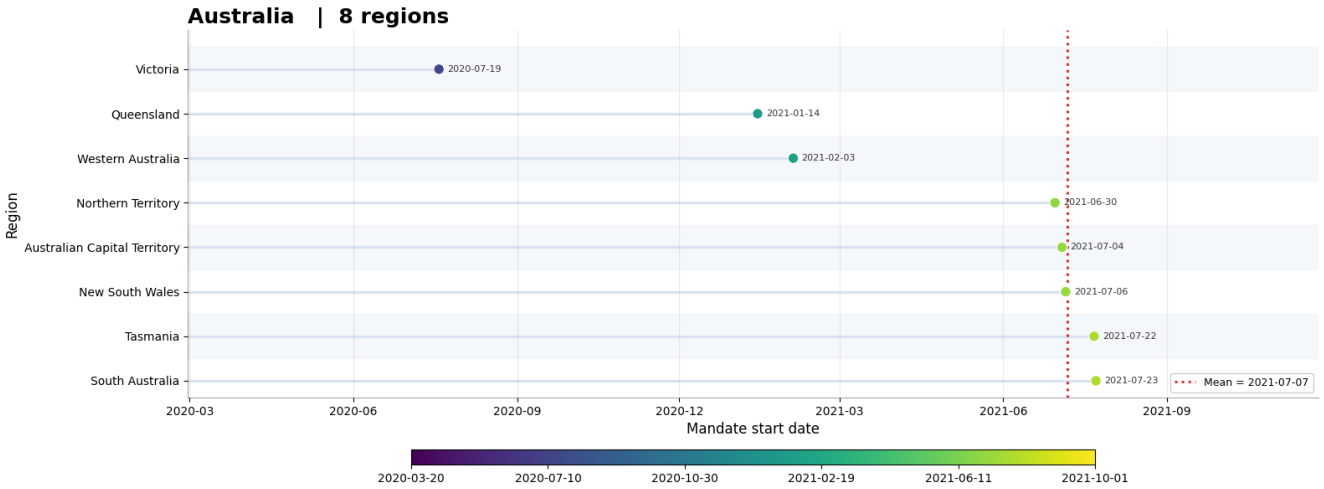

Figure A.2.1: Australia mandate date

A.2.2 Brazil

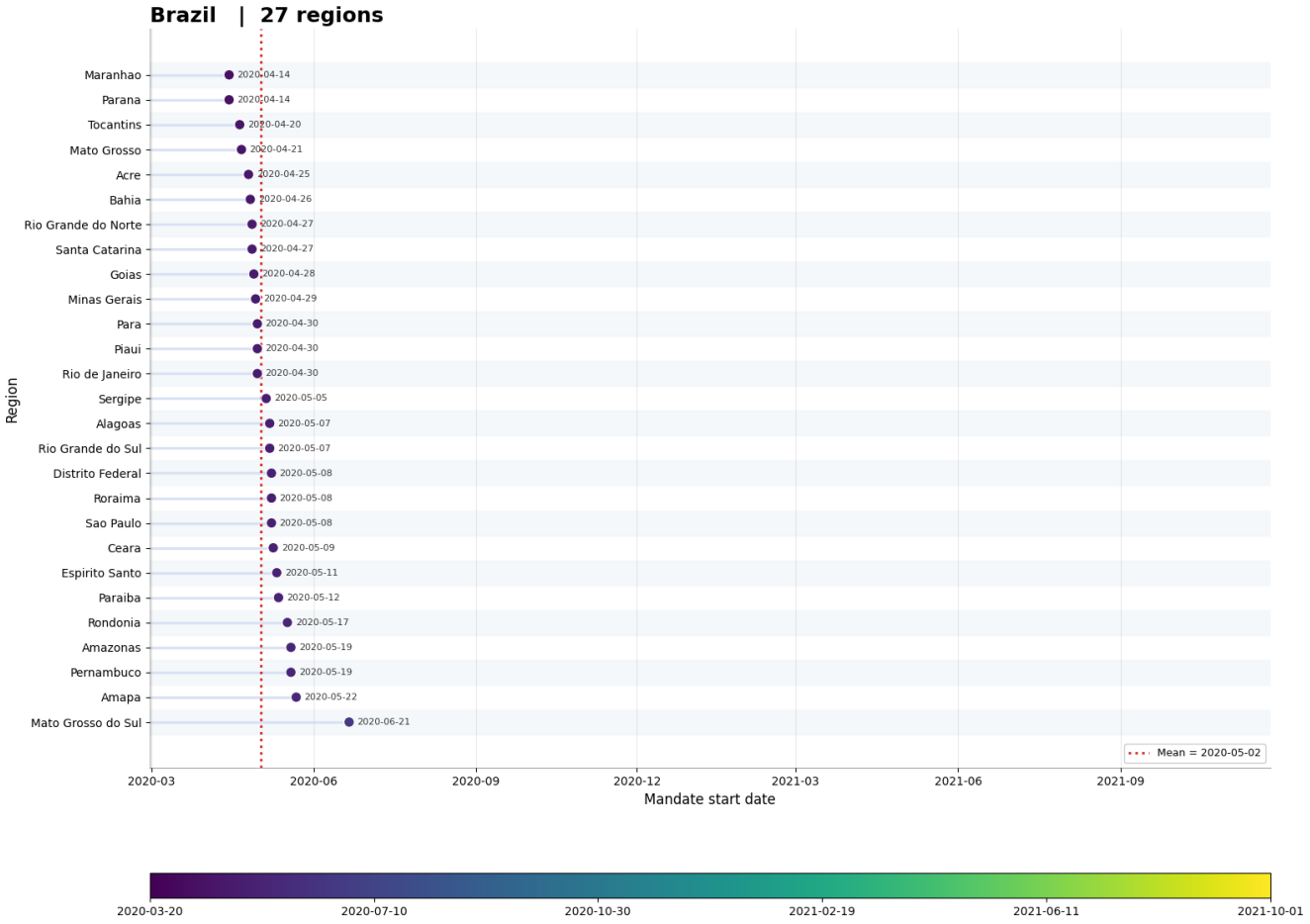

Figure A.2.2: Brazil mandate date

A.2.3 Canada

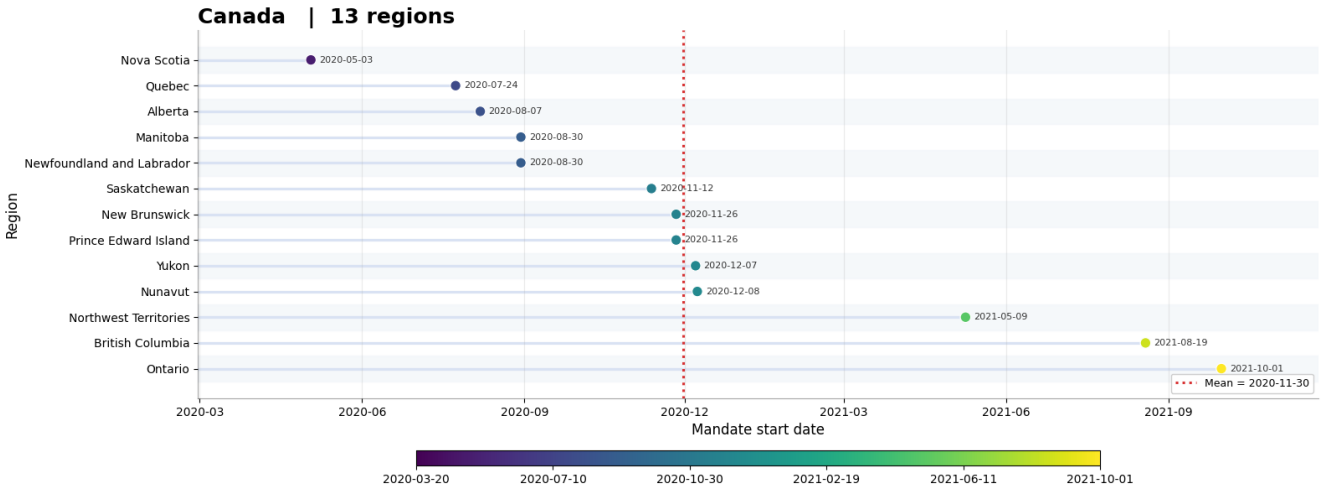

Figure A.2.3: Canada mandate date

A.2.4 United Kingdom

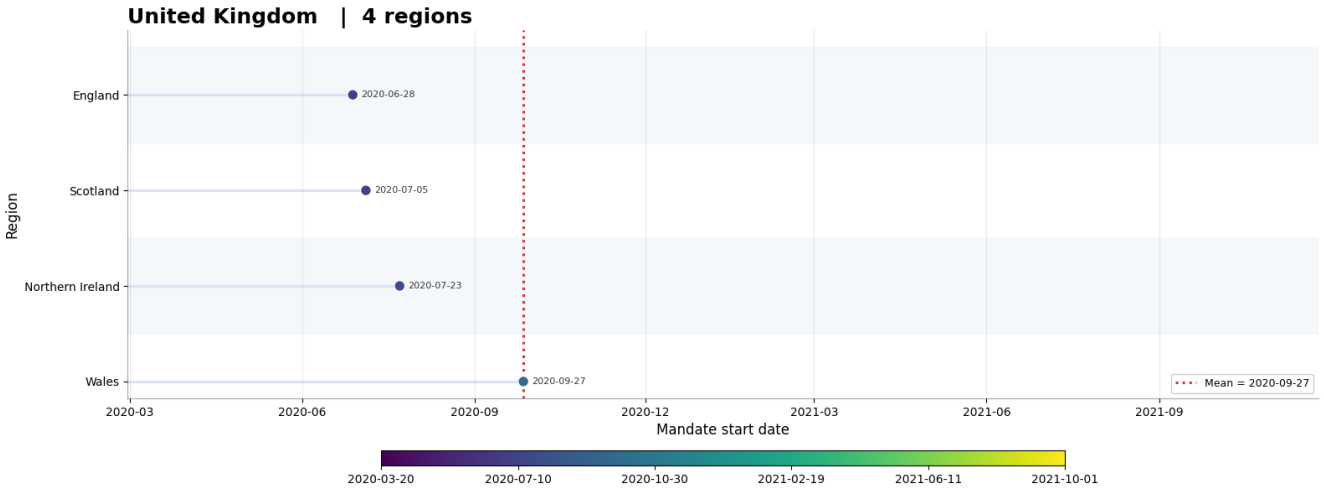

Figure A.2.4: United Kingdom mandate date

A.2.5 India

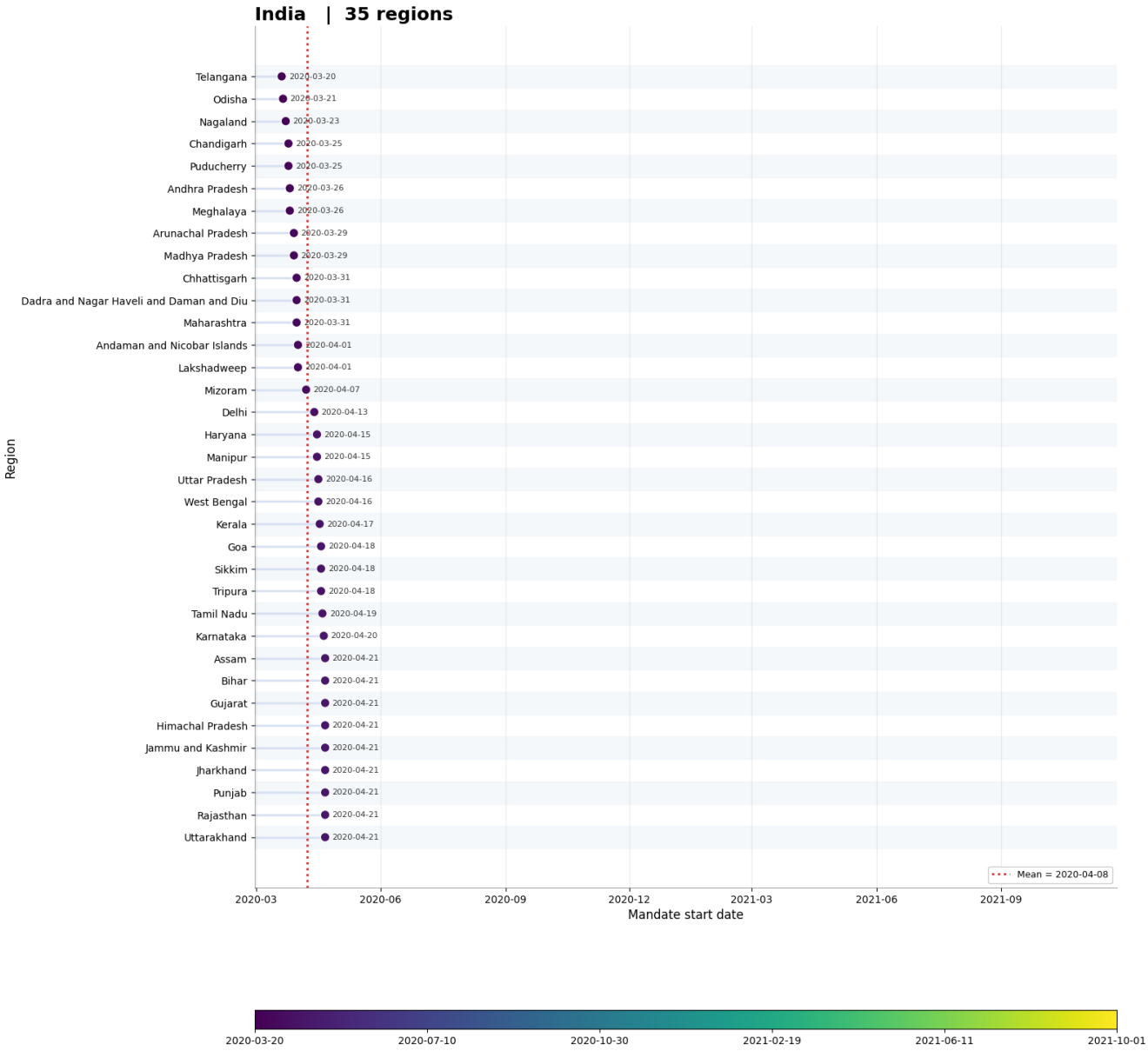

Figure A.2.5: India mandate date

A.2.6 United States

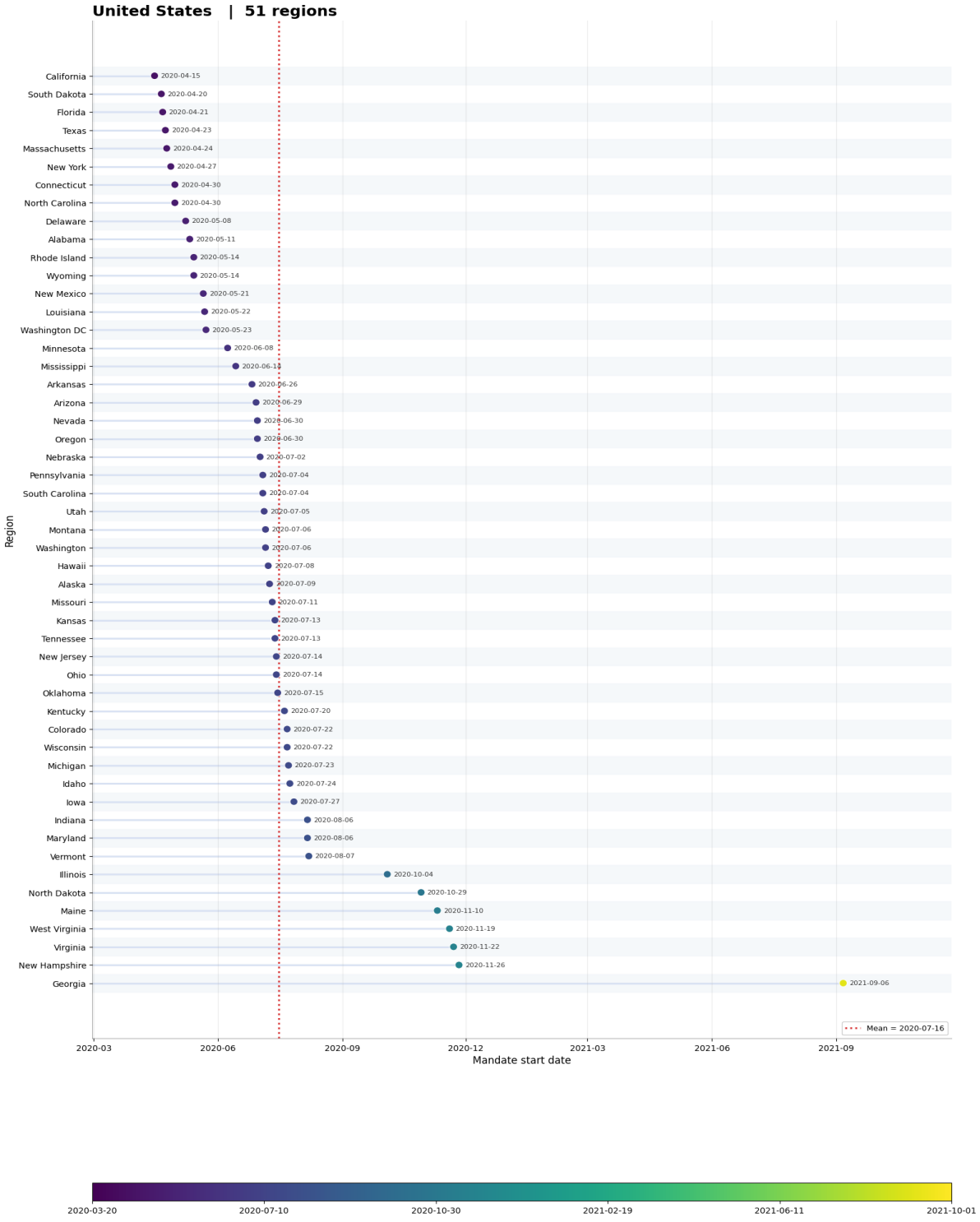

Figure A.2.6: United States mandate date

B Code

Appendix B presents key excerpts and outputs from all source code files cited in the main text. The complete source code and supporting project files are available in the project GitHub repository (Zhao 2026).

B.1 001 Uniform column format.ipynb - Cell 3 - Reading csv

This code defines a helper function that reads a sample from each CSV file using the available encodings. It stores the imported data and creates a summary table containing the data source, country information, filename, detected encoding, and number of columns.

```
# 2. Read all CSV
def read_csv_sample(path):
    for encoding in ENCODINGS:
        try:
            df = pd.read_csv(path,encoding=encoding,nrows=SAMPLE_ROWS,na_values=
            [" ", "__NA__"],low_memory=False)
            return df, encoding
        except UnicodeDecodeError:
            continue
        except Exception:
            return None, encoding
    return None, None

rows = []
raw_data = {}

for source, country_code, path in DATA_FILES:
    df, encoding = read_csv_sample(path)
    raw_data[(source, country_code)] = df
    column_count = df.shape[1]
    rows.append({"source": source, "country_code": country_code,
    "country_name": COUNTRY_NAMES[country_code], "filename": path.name,
    "encoding": encoding,"column_count": column_count})

encoding_check = pd.DataFrame(rows)
```

Figure B.1 summarises the encoding and column-count checks for the sampled CSV files. The results show that the United Kingdom CSV file uses `cp1252` encoding rather than `utf-8-sig`, so it requires separate handling during data import.

	source	country_code	country_name	filename	encoding	column_count
0	OxCGRT	AUS	Australia	OxCGRT_AUS_latest.csv	utf-8-sig	61
1	OxCGRT	BRA	Brazil	OxCGRT_BRA_latest LEGACY.csv	utf-8-sig	61
2	OxCGRT	CAN	Canada	OxCGRT_CAN_latest.csv	utf-8-sig	61
3	OxCGRT	CHN	China	OxCGRT_CHN_latest.csv	utf-8-sig	61
4	OxCGRT	GBR	United Kingdom	OxCGRT_GBR_latest.csv	utf-8-sig	61
5	OxCGRT	IND	India	OxCGRT_IND_latest.csv	utf-8-sig	61
6	OxCGRT	USA	United States	OxCGRT_USA_latest.csv	utf-8-sig	61
7	YouGov	AUS	Australia	australia.csv	utf-8-sig	513
8	YouGov	BRA	Brazil	brazil.csv	utf-8-sig	310
9	YouGov	CAN	Canada	canada.csv	utf-8-sig	512
10	YouGov	CHN	China	china.csv	utf-8-sig	266
11	YouGov	GBR	United Kingdom	united-kingdom.csv	cp1252	528
12	YouGov	IND	India	india.csv	utf-8-sig	334
13	YouGov	USA	United States	united-states.csv	utf-8-sig	424

Figure B.1: CSV file encoding and column-count check.

B.2 001 Uniform column format.ipynb - Cell 5,6 - Modify key columns

The following code checks whether every OxCGRT and YouGov file contains the columns required by the common preprocessing workflow. It records files that cannot be found and lists any required columns that are missing.

```
def check_missing_columns(data_dict, source, columns):
    rows = []
    for country in COUNTRY_NAMES:
        key = (source, country)
        if key not in data_dict:
            rows.append({"source": source,"country_code": country,
            "country_name": COUNTRY_NAMES[country],
            "column": None,"problem": "file not found"})
            continue
        df = data_dict[key]
        for col in columns:
            if col not in df.columns:
                rows.append({"source": source,"country_code": country,
                "country_name": COUNTRY_NAMES[country],
                "column": col,"problem": "missing column"})
    return pd.DataFrame(rows)
```

```
# Check if contain key columns
raw_missing = pd.concat([check_missing_columns(raw_data,
"OxCGRT", USED_OXCGRT_COLS),
check_missing_columns(raw_data, "YouGov", USED_YOUGOV_COLS),],ignore_index=True)

# Visualise
print("Errors:")
if raw_missing.empty:
    display(pd.DataFrame({"message": ["No missing columns"]}))
else:
    display(raw_missing)
```

Figure B.2.1 presents the resulting audit and shows that the China YouGov file does not contain several required variables, including columns needed for employment and policy-attitude predictors.

Errors:

	source	country_code	country_name	column	problem
0	OxCGRT	BRA	Brazil	H6M_Facial Coverings	missing column
1	YouGov	BRA	Brazil	state	missing column
2	YouGov	CAN	Canada	state	missing column
3	YouGov	CHN	China	state	missing column
4	YouGov	CHN	China	employment_status	missing column
5	YouGov	CHN	China	WCRex1	missing column
6	YouGov	CHN	China	WCRex2	missing column
7	YouGov	GBR	United Kingdom	state	missing column

Figure B.2.1: Wrong columns

The next code block standardises the column names needed for the shared workflow. It renames `H6_Facial Coverings` to `H6M_Facial Coverings` in the Brazil OxCGRT file and renames `region` to `state` in the YouGov files for Brazil, Canada and the United Kingdom. China is then removed from the final country list because its missing columns prevent consistent preprocessing and modelling.

```
# Rename columns
RENAME_MAP = {
    ("OxCGRT", "BRA"): {"H6_Facial Coverings": "H6M_Facial Coverings"},
    ("YouGov", "BRA"): {"region": "state"},
```

```
    ("YouGov", "CAN"): {"region": "state"},
    ("YouGov", "GBR"): {"region": "state"}}

# Delete China
EXCLUDED_COUNTRIES = ["CHN"]
FINAL_COUNTRIES = [country for country in COUNTRY_NAMES if country not in EXCLUDED_COUNTRI
```

Figure B.2.2 shows these renaming and exclusion settings.

	operation	source	country_code	country_name	original_column	new_column
0	rename column	OxCGRT	BRA	Brazil	H6_Facial Coverings	H6M_Facial Coverings
1	rename column	YouGov	BRA	Brazil	region	state
2	rename column	YouGov	CAN	Canada	region	state
3	rename column	YouGov	GBR	United Kingdom	region	state
4	exclude country	All	CHN	China	NaN	NaN

```
Excluded countries: ['CHN']
Included countries: ['AUS', 'BRA', 'CAN', 'GBR', 'IND', 'USA']
```

Figure B.2.2: Correct the key columns

B.3 001_2 State mapping.ipynb - Cell 3,4,5 - Mapping regions

The following code compares the state or region names in the YouGov and OxCGRT files after normalising their text. It identifies names that appear in one dataset but do not have a corresponding match in the other dataset for each country.

```
# Find differences between YouGov and OxCGRT
def check_unmatched_states(yougov_df, oxcgrt_df):
    yougov = pd.DataFrame({"yougov_state": sorted(yougov_df["state"].dropna().unique())})
    oxcgrt = pd.DataFrame({"oxcgrt_region": sorted(oxcgrt_df["RegionName"].dropna().unique

    yougov["key"] = yougov["yougov_state"].map(norm_text)
    oxcgrt["key"] = oxcgrt["oxcgrt_region"].map(norm_text)

    yougov_only = (yougov.loc[~yougov["key"].isin(set(oxcgrt["key"])),
    "yougov_state"].reset_index(drop=True).rename("yougov_state_without_oxcgrt_match"))
    oxcgrt_only = (oxcgrt.loc[~oxcgrt["key"].isin(set(yougov["key"])),
    "oxcgrt_region"].reset_index(drop=True).rename("oxcgrt_region_without_yougov_match"))

    return pd.concat([yougov_only, oxcgrt_only], axis=1)

before_unmatched_tables = {}
```

```python
for code, country_name in COUNTRIES.items():
    print("\n" + "=" * 100)
    print(f"{country_name}")
    print("=" * 100)

    yougov_path = UNIFORM_DIR / f"YouGov_{country_name}.csv"
    oxcgrt_path = UNIFORM_DIR / f"OxCGRT_{country_name}.csv"

    yougov_df = read_csv_clean(yougov_path)
    oxcgrt_df = read_csv_clean(oxcgrt_path)

    unmatched_table = check_unmatched_states(yougov_df, oxcgrt_df)
    before_unmatched_tables[code] = unmatched_table

    if unmatched_table.empty:
        print("Everything ok")
    else:
        print("Error:")
        display(unmatched_table)
```

Figures B.3.1–B.3.4 show the unmatched regional names identified for Canada, the United Kingdom, India and the United States before the state-mapping corrections were applied.

```
====================================================================================================
Canada
====================================================================================================
Error:
```

	yougov_state_without_oxcgrt_match	oxcgrt_region_without_yougov_match
0	British Columbia / Colombie Britanique	British Columbia
1	New Brunswick / Nouveau-Brunswick	New Brunswick
2	Newfoundland & Labrador / Terre-Neuve-et-Labrador	Newfoundland and Labrador
3	Northwest Territories / Territoires du Nord-Ouest	Northwest Territories
4	Nova Scotia / Nouvelle-Écosse	Nova Scotia
5	Prince Edward Island / Île-du-Prince-Édouard	Prince Edward Island
6	Quebec / Québec	Quebec

Figure B.3.1: Canada no mapped regions

United Kingdom

Error:

	yougov_state_without_oxcgrt_match	oxcgrt_region_without_yougov_match
0	East Midlands	England
1	East of England	NaN
2	London	NaN
3	North East	NaN
4	North West	NaN
5	South East	NaN
6	South West	NaN
7	West Midlands	NaN
8	Yorkshire and the Humber	NaN

Figure B.3.2: United Kingdom no mapped regions

India

Error:

	yougov_state_without_oxcgrt_match	oxcgrt_region_without_yougov_match
0	Andaman & Nicobar Islands	Andaman and Nicobar Islands
1	Dadra & Nagar Haveli	Dadra and Nagar Haveli and Daman and Diu
2	Daman & Diu	Jammu and Kashmir
3	Jammu & Kashmir	Ladakh

Figure B.3.3: India no mapped regions

United States

Error:

	yougov_state_without_oxcgrt_match	oxcgrt_region_without_yougov_match
0	District of Columbia	Washington DC

Figure B.3.4: United States no mapped region

The following mapping corrects the unmatched regional names identified above for Canada, the United Kingdom, India and the United States. It standardises the relevant YouGov state names

so that they correspond to the OxCGRT region names. The Indian region of Ladakh is removed because it cannot be mapped consistently between the two datasets.

```
STATE_MAPPING = {

    # Correct some naming errors (Canada)
    "Canada": {
        "British Columbia / Colombie Britanique": "British Columbia",
        "British Columbia / Colombie Britannique": "British Columbia",
        "New Brunswick / Nouveau-Brunswick": "New Brunswick",
        "Newfoundland & Labrador / Terre-Neuve-et-Labrador": "Newfoundland and Labrador",
        "Northwest Territories / Territoires du Nord-Ouest": "Northwest Territories",
        "Nova Scotia / Nouvelle-Écosse": "Nova Scotia",
        "Prince Edward Island / Île-du-Prince-Édouard": "Prince Edward Island",
        "Quebec / Québec": "Quebec"},

    # Correct some naming errors (UK)
    "United Kingdom": {
        "East Midlands": "England",
        "East of England": "England",
        "London": "England",
        "North East": "England",
        "North West": "England",
        "South East": "England",
        "South West": "England",
        "West Midlands": "England",
        "Yorkshire and the Humber": "England"},

    # Correct some naming errors (India)
    "India": {
        "Andaman & Nicobar Islands": "Andaman and Nicobar Islands",
        "Dadra & Nagar Haveli": "Dadra and Nagar Haveli and Daman and Diu",
        "Daman & Diu": "Dadra and Nagar Haveli and Daman and Diu",
        "Jammu & Kashmir": "Jammu and Kashmir",},

    # Correct some naming errors (US)
    "United States": {"District of Columbia": "Washington DC"}}


# Delete regions that cannot be mapped (India)
DROP_OXCGRT_REGIONS = {"India": ["Ladakh"]}
```

B.4 002 Data Clean.ipynb - Cell 9 - Remove unusable columns and variables

The following code loads the YouGov and OxCGRT files for each retained country and stores copies of the data in separate dictionaries. It calculates the missing-value rate for every YouGov column, retains only columns with no more than 30% missing values, and removes `cantril_ladder`, `qweek`, `weight` and `household_children`. The final list of retained variables for each country is saved in `retained_columns_by_country` for subsequent modelling.

```
# Create 3 dictionaries
raw_yougov_data = {}                          # Yougov data
raw_oxcgrt_data = {}                          # OxCGRT data
retained_columns_by_country = {}              # Variable names retained

# Read
for code, country_name in COUNTRIES.items():

    yougov_path = UNIFORM_DIR / f"YouGov_{country_name}.csv"
    oxcgrt_path = UNIFORM_DIR / f"OxCGRT_{country_name}.csv"

    yougov_df = read_csv_clean(yougov_path)
    oxcgrt_df = read_csv_clean(oxcgrt_path)

    raw_yougov_data[code] = yougov_df.copy()
    raw_oxcgrt_data[code] = oxcgrt_df.copy()

    # FilterVariablesand
    missing_rate = yougov_df.isna().mean()

    # Retain variables (attrition rate <= 30%)
    retained_cols = missing_rate[missing_rate <= 0.3].index.tolist()

    # If the 4 columns are retained,
    # this will result in a large number of missing values in the sample
    retained_cols = [col for col in retained_cols if col not in
    ["cantril_ladder","qweek","weight","household_children"]]

    # Save
    retained_columns_by_country[code] = retained_cols
```

B.5 002 Data Clean.ipynb - Cell 5 - Recode

```
# Recode risk perception
def recode_agreement_value(x):
    if pd.isna(x):
```

```
        return np.nan
    x = str(x).strip()
    agreement_map = {          # Some Formatting Issues
        "7 - Agree": 7,
        "7 - Agree": 7,
        "7 - Agree": 7,
        "7": 7,
        "6": 6,
        "5": 5,
        "4": 4,
        "3": 3,
        "2": 2,
        "1 - Disagree": 1,
        "1 - Disagree": 1,
        "1 - Disagree": 1,
        "1": 1}
    return agreement_map.get(x, np.nan)


# Recode frequency of protective behaviors
def recode_frequency_value(x):
    if pd.isna(x):
        return np.nan
    x = str(x).strip()

    frequency_map = {          # Definition scale
    "Always": 5,
    "Frequently": 4,
    "Sometimes": 3,
    "Rarely": 2,
    "Not at all": 1}

    return frequency_map.get(x, np.nan)
```

This code converts the agreement-scale risk-perception responses into ordered scores from 1 to 7 and converts protective-behaviour frequency responses into ordered scores from 1 to 5. Missing values and unrecognised responses are returned as missing values.

B.6 002 Data Clean.ipynb - Cell 10 - Outcome definitions

```
# Common columns retained in all countries
common_columns = sorted
(set.intersection(*[set(cols) for cols in retained_columns_by_country.values()]))
# "i12_health_xxx" Common columns retained in all countries
```

```
common_i12_cols = sort_i12_columns
([col for col in common_columns if col.startswith("i12_health_")])


# Face mask outcome
FACE_MASK_OUTCOME_COLS = ["i12_health_1"]
missing_face_mask_cols =
[col for col in FACE_MASK_OUTCOME_COLS if col not in common_i12_cols]
if len(missing_face_mask_cols) > 0:
    raise ValueError("error")


# Cross-national outcome item sets
COMMON_I12_OUTCOME_COLS = common_i12_cols

# No mask protective behaviour
NON_MASK_PROTECTIVE_BEHAVIOUR_OUTCOME_COLS =
[col for col in COMMON_I12_OUTCOME_COLS if col not in FACE_MASK_OUTCOME_COLS]

# Overall protective behaviour
OVERALL_PROTECTIVE_BEHAVIOUR_OUTCOME_COLS = ( COMMON_I12_OUTCOME_COLS.copy() )


# Display
outcome_definition_table = pd.DataFrame({
    "outcome":
    ["face_mask_behaviour_binary",
    "non_mask_protective_behaviour_binary",
    "overall_protective_behaviour_binary"],

    "columns_used": [", ".join(FACE_MASK_OUTCOME_COLS),
    ",".join(NON_MASK_PROTECTIVE_BEHAVIOUR_OUTCOME_COLS),
    ",".join(OVERALL_PROTECTIVE_BEHAVIOUR_OUTCOME_COLS)],
    "number_of_columns": [len(FACE_MASK_OUTCOME_COLS),
    len(NON_MASK_PROTECTIVE_BEHAVIOUR_OUTCOME_COLS),
    len(OVERALL_PROTECTIVE_BEHAVIOUR_OUTCOME_COLS)]})

display(outcome_definition_table)
```

The code identifies the protective-behaviour variables retained across all countries and defines the item sets used for the face-mask, non-mask protective-behaviour and overall protective-behaviour outcomes. Figure B.6.1 shows the resulting definition table. The two outcomes are "`Face Mask Wearing`" and "`Overall Protective Behaviour`". "Non-Mask Protective Behaviour" is the independent variable for "Face Mask Wearing".

	outcome	columns_used	number_of_columns
0	face_mask_behaviour_binary	i12_health_1	1
1	non_mask_protective_behaviour_binary	i12_health_2, i12_health_3, i12_health_4, i12_...	13
2	overall_protective_behaviour_binary	i12_health_1, i12_health_2, i12_health_3, i12_...	14

Figure B.6.1: Outcome definitions and retained protective-behaviour items

B.7 002 Data Clean.ipynb - Cell7

```
# Convert behaviour scale to binary compliance outcome
# Protective behaviors frequency is (1~5)
# (1 = Not at all   2 = Rarely   3 = Sometimes   4 = Frequently   5 = Always)
def binary_from_scale(series, threshold=4):

    # if Frequency >= 4 → 1 ; if Frequency < 4 → 0
    return np.where(series.notna(),np.where(series >= threshold, 1, 0),np.nan)


# Create face mask, non-mask and overall behaviour outcomes
def add_behaviour_outcomes(df):
    out = df.copy()

    # Face mask behaviour scale
    out["face_mask_behaviour_scale"] =
    out[FACE_MASK_OUTCOME_COLS].median(axis=1, skipna=False)

    # Face mask behaviour binary
    out["face_mask_behaviour_binary"] =
    binary_from_scale(out["face_mask_behaviour_scale"])

    # No mask protective behaviour scale
    out["non_mask_protective_behaviour_scale"] =
    out[NON_MASK_PROTECTIVE_BEHAVIOUR_OUTCOME_COLS].median(axis=1, skipna=False)

    # No mask protective behaviour binary
    out["non_mask_protective_behaviour_binary"] =
    binary_from_scale(out["non_mask_protective_behaviour_scale"])

    # Overall protective behaviour scale
    out["overall_protective_behaviour_scale"] =
    out[OVERALL_PROTECTIVE_BEHAVIOUR_OUTCOME_COLS].median(axis=1, skipna=False)

    # Overall protective behaviour binary
    out["overall_protective_behaviour_binary"] =
```

```
    binary_from_scale(out["overall_protective_behaviour_scale"])

    return out
```

The function first converts each protective-behaviour scale into a binary compliance outcome. Scores of 4 or 5, corresponding to "Frequently" or "Always", are classified as compliant and coded as 1, while scores below 4 are coded as 0. Missing scale values remain missing rather than being assigned to either class.

The second function creates three respondent-level outcomes. The face-mask scale is calculated from the designated face-mask item, the non-mask protective-behaviour scale is the median of the retained protective-behaviour items excluding face-mask wearing, and the overall protective-behaviour scale is the median across all retained protective-behaviour items. Because `skipna=False` is used, each median is calculated only when all items required for that score are observed; otherwise, the resulting scale and binary outcome remain missing. The non-mask score can therefore be used as a predictor in the face-mask task, whereas it should not be included as a predictor in the overall protective-behaviour task because its component items also contribute to the overall outcome.

B.8 002 Data Clean.ipynb - Cells 4,8,11 - Data Clean

The following selected code excerpts show the key operations used to handle missing values, construct the comorbidity variable and create the relative survey-time variable without reproducing all supporting code from the three notebook cells.

```
# Return columns treated as categorical variables
def get_categorical_columns(df):
    return df.select_dtypes(include=["object","category","bool","string"]).columns.tolist()


# Retain categorical missing values as an explicit category
def fill_categorical_missing_with_NA(df):
    out = df.copy()
    categorical_cols = get_categorical_columns(out)

    if len(categorical_cols) > 0:
        out[categorical_cols] = out[categorical_cols].fillna("N/A")
    return out


# Drop rows with missing values in numeric or datetime columns
def drop_rows_with_numeric_or_datetime_missing(df):
    out = df.copy()
    numeric_or_datetime_cols = out.select_dtypes
    (include=["number", "datetime64[ns]"]).columns.tolist()
```

```
    rows_before = out.shape[0]

    if len(numeric_or_datetime_cols) > 0:
        out = out.dropna(subset=numeric_or_datetime_cols).copy()

    rows_after = out.shape[0]
    rows_dropped = rows_before - rows_after

    return out, rows_dropped, numeric_or_datetime_cols


# Collapse the original d1_health_* variables into one comorbidity variable
def collapse_comorbidity_variables(df):
    out = df.copy()
    d1_cols =
    [ col for col in out.columns if col.startswith("d1_health_") ]

    if len(d1_cols) == 0:
        return out

    out["d1_comorbidities"] = "N/A"

    disease_cols =
    [col for col in d1_cols if col not in ["d1_health_98", "d1_health_99"]]

    if len(disease_cols) > 0:
        has_comorbidity = ( out[disease_cols].astype(str).eq("Yes").any(axis=1))
        out.loc[ has_comorbidity, "d1_comorbidities"] = "Yes"

    if "d1_health_99" in out.columns:
        out.loc[out["d1_health_99"].astype(str).eq("Yes"),"d1_comorbidities"] = "No"

    if "d1_health_98" in out.columns:
        out.loc[
            out["d1_health_98"].astype(str).eq("Yes"),
           "d1_comorbidities" ] = "Prefer_not_to_say"

    out = out.drop(columns=d1_cols)
    return out


# Create a relative survey-time variable using 14-day intervals
def add_week_number(df):
    out = df.copy()
    start_date = out["endtime"].min()
    out["week_number"] =
```

```
    (((out["endtime"] - start_date).dt.days // 14) + 1)
    return out


# Selected processing sequence from the country-level cleaning loop
df = add_behaviour_outcomes(df)

df, rows_dropped_numeric_or_datetime, numeric_or_datetime_cols_checked =
    (drop_rows_with_numeric_or_datetime_missing(df))

df = fill_categorical_missing_with_NA(df)
df = collapse_comorbidity_variables(df)
df = add_week_number(df)
```

After the behaviour outcomes are created, the code identifies all numeric and datetime variables and removes any row with a missing value in those columns. Missing values in categorical variables are not used to remove respondents; instead, they are replaced with the explicit category `N/A`, allowing these responses to remain in the dataset and later be dummy encoded.

The original disease-specific `d1_health_*` columns are then combined into a single categorical variable. A respondent is assigned `Yes` when at least one disease-specific item is marked `Yes`, while the original responses representing no comorbidity and unwillingness to disclose health information are retained as `No` and `Prefer_not_to_say`. Respondents without a recorded category remain `N/A`, and the original disease-specific columns are removed.

Finally, `week_number` expresses survey timing relative to the first retained response date within each country. The difference between each respondent's completion date and that starting date is divided into consecutive 14-day intervals, with the first interval coded as 1.

B.9 002 Data Clean.ipynb - Cells 3,6 - Policy Period

The following selected code excerpts show how the regional face-covering policy dates and the respondent-level policy-period variable were constructed without reproducing the unrelated helper and audit code from the notebook.

```
# Settings used to identify sustained face-covering policy periods
ROLLING_DAYS = 14
MANDATE_THRESHOLD = 2.5


# Parse the survey completion date
def parse_endtime(x):
    return pd.to_datetime(x, dayfirst=True, errors="coerce")


# Compute the policy start date for each subnational region
```

```
def compute_mandate_dates(oxcgrt_df):
    df = oxcgrt_df.copy()

    # Convert YYYYMMDD policy dates and the face-covering indicator
    df["policy_date"] = pd.to_datetime(
        df["Date"].astype(str),
        format="%Y%m%d",
        errors="coerce")
    df["H6M_Facial Coverings"] = pd.to_numeric(
        df["H6M_Facial Coverings"],
        errors="coerce")

    df = df.dropna( subset=["policy_date", "H6M_Facial Coverings"]).copy()
    df = df.sort_values("policy_date")

    region_rows = []
    region_df = df[df["RegionName"].notna()].copy()

    for region_name, g in region_df.groupby("RegionName"):
        g = g.sort_values("policy_date").copy()

        # A value is calculated only when all 14 daily observations are present
        g["rolling_h6"] = (
            g["H6M_Facial Coverings"].rolling(ROLLING_DAYS,
            min_periods=ROLLING_DAYS).mean())

        valid_rolling = g[g["rolling_h6"].notna()].copy()

        if len(valid_rolling) == 0:
            continue

        hit = valid_rolling[valid_rolling["rolling_h6"] >= MANDATE_THRESHOLD]

        if len(hit) > 0:
            mandate_start_date = hit["policy_date"].iloc[0]
            mandate_rule = "threshold"
        else:
            max_idx = valid_rolling["rolling_h6"].idxmax()
            mandate_start_date = valid_rolling.loc[
                max_idx, "policy_date" ]
            mandate_rule = "max_rolling"

        region_rows.append({
            "state": region_name,
            "mandate_start_date": mandate_start_date,
            "mandate_rule": mandate_rule})
```

```
    region_mandates = pd.DataFrame(
        region_rows,
        columns=["state", "mandate_start_date", "mandate_rule"])
    return region_mandates


# Add the respondent-level policy-period indicator
def add_mandate_period(yougov_df, region_mandates):
    out = yougov_df.copy()
    out["endtime"] = parse_endtime(out["endtime"])

    out = out.merge(region_mandates, on="state", how="left")

    out["within_mandate_period"] = np.nan
    valid = (out["endtime"].notna() & out["mandate_start_date"].notna())

    out.loc[valid, "within_mandate_period"] = (
        out.loc[valid, "endtime"]
        >= out.loc[valid, "mandate_start_date"]).astype(int)

    out = out.drop(
        columns=["mandate_start_date", "mandate_rule"],
        errors="ignore")
    return out
```

The code first converts the OxCGRT policy date from `YYYYMMDD` format to a calendar date and converts `H6M_Facial Coverings` to a numeric variable. It then processes each subnational region separately and calculates a 14-day rolling mean. The use of `min_periods=ROLLING_DAYS` means that a rolling value is retained only when all 14 daily observations are available.

For each region, the first date on which the rolling mean reaches the policy threshold is selected as the policy start date. If the threshold is never reached, the date with the maximum valid rolling mean is used. These regional dates are then merged with the YouGov data. Respondents surveyed on or after their region's policy start date are coded as 1 for `within_mandate_period`, and respondents surveyed before that date are coded as 0. Cases without a valid survey date or matched regional policy date remain missing.

B.10 002 Data Clean.ipynb - Cells 6 - Threshold

This code determines the start date of the face-covering policy for each subnational region. It first converts the OxCGRT date and face-covering indicator into appropriate date and numeric formats, removes invalid observations, and orders the records chronologically. A 14-day rolling mean of the `H6M_Facial Coverings` indicator is then calculated separately for each region, with a value produced only when all 14 daily observations are available.

```
# Compute mandate date
def compute_mandate_dates(oxcgrt_df):

    df = oxcgrt_df.copy()                                                    # Copy

    df["policy_date"] = pd.to_datetime
    ( df["Date"].astype(str), format="%Y%m%d", errors="coerce")              # Convert date

    df["H6M_Facial Coverings"] =
    pd.to_numeric( df["H6M_Facial Coverings"],errors="coerce")               # Convert H6M_Facial

    df = df.dropna(subset=["policy_date", "H6M_Facial Coverings"]).copy()
    df = df.sort_values("policy_date")                                       # Sort


    # 1 Region mandate dates
    region_rows = []
    region_df = df[df["RegionName"].notna()].copy()

    for region_name, g in region_df.groupby("RegionName"):
        g = g.sort_values("policy_date").copy()

        # 14-day rolling average to identify "ongoing" mask policy phases
        g["rolling_h6"] = (g["H6M_Facial Coverings"]
        .rolling(ROLLING_DAYS, min_periods=ROLLING_DAYS).mean())

        # Keep valid rolling average values
        valid_rolling = g[g["rolling_h6"].notna()].copy()

        if len(valid_rolling) == 0:
            continue

        # 14-day threshold
        hit = valid_rolling[valid_rolling["rolling_h6"] >= MANDATE_THRESHOLD]

        # First date reaching the threshold
        if len(hit) > 0:
            mandate_start_date = hit["policy_date"].iloc[0]
            mandate_rule = "threshold"
        else:
            # First date reaching the maximum rolling average
            max_idx = valid_rolling["rolling_h6"].idxmax()
            mandate_start_date = valid_rolling.loc[max_idx, "policy_date"]
            mandate_rule = "max_rolling"

        region_rows.append({"state": region_name,
```

```
        "mandate_start_date": mandate_start_date,
        "mandate_rule": mandate_rule})

    region_mandates = pd.DataFrame(region_rows,
    columns=["state", "mandate_start_date", "mandate_rule"])

    return region_mandates
```

The mandate start date is defined as the first date on which the 14-day rolling mean reaches the specified policy threshold. When a region never reaches this threshold, the date with the highest observed 14-day rolling mean is used instead. The resulting table records each region, its estimated mandate start date, and whether the date was identified using the threshold rule or the maximum rolling-mean fallback rule.

B.11 002 Data Clean.ipynb - Cells 6 - Mandate Period

```
# Add the mandate period (0/1)
def add_mandate_period(yougov_df,region_mandates):

    # Copy
    out = yougov_df.copy()

    # Analyze the survey completion date
    out["endtime"] = parse_endtime(out["endtime"])

    # Merge
    out = out.merge(region_mandates,on="state",how="left")

    out["within_mandate_period"] = np.nan
    valid = out["endtime"].notna() & out["mandate_start_date"].notna()

    # Categories
    out.loc[valid, "within_mandate_period"] =
    (out.loc[valid, "endtime"] >= out.loc[valid, "mandate_start_date"]).astype(int)

     # Delete unnecessary columns
    out = out.drop(columns=["mandate_start_date", "mandate_rule"],errors="ignore")
    return out
```

This code creates the binary `within_mandate_period` variable for each respondent. It converts the survey completion date into a valid date format, merges the region-specific mandate start date into the YouGov data using `state`, and compares the two dates. Respondents surveyed on or after

the mandate start date are coded as 1, while those surveyed before it are coded as 0. Cases with a missing survey date or mandate start date remain missing, and the temporary mandate-related columns are removed after the variable is created.

B.12 002 Data Clean.ipynb - Cells 5,11 and 003 Column Names update.ipynb - Cells 3,4 - Feature Encoding

The following code shows the key steps used to convert categorical predictors into numerical dummy variables while keeping the binary outcome columns unchanged. It also illustrates how the encoded column names were converted into labels that were easier to interpret in the modelling results.

```
# Keep binary outcomes as target variables
TARGET_COLUMNS = [
    "face_mask_behaviour_binary",
    "overall_protective_behaviour_binary"]

# Identify categorical predictors only
categorical_cols = [
    col for col in df.select_dtypes(
        include=["object", "category", "bool"]).columns
    if col not in TARGET_COLUMNS]

# One-hot encode categories and use the first category as the reference
encoded_df = pd.get_dummies(
    df,
    columns=categorical_cols,
    drop_first=True,
    dtype=int)

# Convert selected dummy-variable prefixes into readable labels
READABLE_PREFIXES = {
    "state_": "State: ",
    "employment_": "Employment: ",
    "isolation_willingness_": "Isolation Willingness: "}

def readable_column_name(column):
    for prefix, label in READABLE_PREFIXES.items():
        if column.startswith(prefix):
            category = column[len(prefix):].replace("_", " ")
            return f"{label}{category}"
    return column.replace("_", " ")

encoded_df = encoded_df.rename(columns=readable_column_name)
```

The code identifies categorical predictors, excludes the binary outcomes from encoding, and ap-

plies one-hot encoding with `drop_first=True`. Consequently, one category from each categorical variable is retained as the reference category, while the remaining categories are represented by 0/1 dummy columns. The final renaming step replaces technical dummy-variable names with clearer labels such as `State: New South Wales`, `Employment: Retired` and `Isolation Willingness: Very Willing`.

B.13 004 Split.ipynb - Cells 5,6 - Modelling Tasks

The following selected code illustrates how each country's data was divided into four modelling tasks by combining two policy periods with two binary outcomes.

```
PERIODS = {
    0: "before_mandate",
    1: "after_mandate"}

OUTCOMES = {
    "face_mask": "face_mask_behaviour_binary",
    "overall_protective_behaviour":
        "overall_protective_behaviour_binary"}

tasks = {}

for country_code, country_df in encoded_data.items():
    for period_value, period_name in PERIODS.items():
        period_df = country_df.loc[
            country_df["within_mandate_period"] == period_value
        ].copy()

        for outcome_name, target_col in OUTCOMES.items():
            y = period_df[target_col].astype(int)

            # EXCLUDED_COLUMNS is defined separately for each outcome
            X = period_df.drop(
                columns=EXCLUDED_COLUMNS[outcome_name],
                errors="ignore").copy()

            # Convert Boolean dummy columns to numerical 0/1 values
            bool_cols = X.select_dtypes(include=["bool"]).columns
            X[bool_cols] = X[bool_cols].astype(int)

            # Confirm that every predictor supplied to the model is numeric
            non_numeric = [
                col for col in X.columns
                if not pd.api.types.is_numeric_dtype(X[col])]

            if non_numeric:
```

```
                raise TypeError(
                    f"Non-numeric predictors found: {non_numeric}")

            task_name = (
                f"{country_code}_{period_name}_{outcome_name}")
            tasks[task_name] = {"X": X, "y": y}
```

For each retained country, the code first separates responses into before-mandate and after-mandate subsets. Each period is then paired with the face-mask outcome and the overall protective-behaviour outcome, producing four tasks per country. Boolean dummy variables are converted to integers, and the final check prevents a task from being saved when any non-numeric predictor remains.

B.14 004 Split.ipynb - Cells 3,6 - Feature Exclusion

The following code shows the outcome-specific feature-exclusion rules used before modelling. These rules remove organisational variables and outcome-derived variables while retaining the non-mask protective behaviour score only for the face-mask task.

```
ORGANISATIONAL_COLUMNS = [
    "respondent_id",
    "endtime",
    "within_mandate_period"]

BINARY_OUTCOME_COLUMNS = [
    "face_mask_behaviour_binary",
    "non_mask_protective_behaviour_binary",
    "overall_protective_behaviour_binary"]

BEHAVIOUR_SCALE_COLUMNS = [
    "face_mask_behaviour_scale",
    "non_mask_protective_behaviour_scale",
    "overall_protective_behaviour_scale"]

def excluded_columns_for_task(outcome_name):
    excluded = set(
        ORGANISATIONAL_COLUMNS
        + BINARY_OUTCOME_COLUMNS
        + BEHAVIOUR_SCALE_COLUMNS)

    if outcome_name == "face_mask":
        # This score excludes the mask item and can be used as a predictor
        excluded.remove(
            "non_mask_protective_behaviour_scale")
```

```
    elif outcome_name != "overall_protective_behaviour":
        raise ValueError(f"Unknown outcome: {outcome_name}")

    return sorted(excluded)


EXCLUDED_COLUMNS = {
    "face_mask":
        excluded_columns_for_task("face_mask"),
    "overall_protective_behaviour":
        excluded_columns_for_task(
            "overall_protective_behaviour")}
```

Respondent identifiers, survey dates and the mandate-period indicator are removed because they organise the data rather than represent substantive predictors. All binary outcome columns and behaviour scales are also excluded to prevent the model from receiving information derived directly from an outcome. The only exception is `non_mask_protective_behaviour_scale` in the face-mask task, because this score is calculated without the face-mask item. It is excluded from the overall protective-behaviour task because its component items also contribute to that outcome.

B.15 004 Split.ipynb - Cells 3,5,6 - Train-Test Split

The following code shows the common 80/20 splitting rule applied to every country-period-outcome task and the condition used to determine whether stratification was possible.

```
from sklearn.model_selection import train_test_split

TEST_SIZE = 0.20
RANDOM_SEED = 1948883

def split_task(X, y):
    class_counts = y.value_counts(dropna=False)

    # Stratify only when both binary classes have at least two cases
    use_stratification = (
        len(class_counts) == 2
        and class_counts.min() >= 2)

    stratify_target = y if use_stratification else None

    return train_test_split(
        X,
        y,
        test_size=TEST_SIZE,
        random_state=RANDOM_SEED,
```

```
        stratify=stratify_target)


for task_name, task_data in tasks.items():
    X_train, X_test, y_train, y_test = split_task(
        task_data["X"],
        task_data["y"])

    X_train.to_csv(
        SPLIT_DIR / f"{task_name}_X_train.csv",
        index=False)
    X_test.to_csv(
        SPLIT_DIR / f"{task_name}_X_test.csv",
        index=False)
    y_train.to_csv(
        SPLIT_DIR / f"{task_name}_y_train.csv",
        index=False)
    y_test.to_csv(
        SPLIT_DIR / f"{task_name}_y_test.csv",
        index=False)
```

Each task is divided into 80% training data and 20% test data using the fixed random seed 1948883. Stratification is used only when both outcome classes contain at least two observations, which helps preserve the class distribution in the two subsets. Four files are produced for every task: `X_train`, `X_test`, `y_train` and `y_test`. Model fitting and hyperparameter tuning use only the training files, while the held-out test files remain reserved for final evaluation.

B.16 005 Random Forest (Model).ipynb and 006 XGBoost (Model).ipynb - Cells 4 - RandomOverSampler

The Random Forest pipeline first applies `RandomOverSampler` to balance the class distribution in the training data and then fits a Random Forest classifier. The classifier consists of 250 decision trees, with a fixed random seed for reproducibility and parallel processing enabled to improve computational efficiency.

```
# Random Forest Pipeline
def build_pipeline():
    return Pipeline(steps=[("ros",
    RandomOverSampler(random_state=RANDOM_STATE)),

    # Processing class imbalance
    ("model",
    RandomForestClassifier
    ( n_estimators=250,
    random_state=RANDOM_STATE,
    n_jobs=-1,))])          # Train model
```

The XGBoost pipeline similarly applies `RandomOverSampler` before fitting the classification model, thereby reducing the influence of class imbalance during training. The model uses 250 boosting iterations with a binary logistic objective, log-loss evaluation metric, and the histogram-based tree-building method for efficient computation.

```
# XGBoost Pipeline
def Build_pipeline():

    # Processing class imbalance
    return Pipeline(steps=[("ros", RandomOverSampler(random_state=RANDOM_STATE)),
                           ("model", XGBClassifier(n_estimators=250,
                           objective="binary:logistic",
                           eval_metric="logloss",
                           random_state=RANDOM_STATE,
                           n_jobs=-1,
                           tree_method="hist",))])          # Train model
```

B.17 005 Random Forest (Model).ipynb and 006 XGBoost (Model).ipynb - Cells 3,4,5 - Hyperparameter Tuning

The following code shows the common hyperparameter-tuning procedure used for both classifiers. The number of stratified cross-validation folds was determined from the minority-class count in each training set.

```
from sklearn.model_selection import StratifiedKFold, RandomizedSearchCV

RANDOM_STATE = 1948883
N_SPLITS = 5
N_ITER_SEARCH = 40

def build_search(y_train, pipeline, parameter_grid):
    min_class_count = y_train.value_counts().min()

    # A stratified split requires at least two cases in each class
    if min_class_count < 2:
        raise ValueError("Not enough minority-class observations")

    # Use at most five folds and reduce the number when necessary
    n_splits = min(N_SPLITS, int(min_class_count))
    cv = StratifiedKFold(
        n_splits=n_splits,
        shuffle=True,
        random_state=RANDOM_STATE)
```

```
    return RandomizedSearchCV(
        estimator=pipeline,
        param_distributions=parameter_grid,
        n_iter=N_ITER_SEARCH,
        scoring="roc_auc",
        cv=cv,
        random_state=RANDOM_STATE,
        refit=True,
        n_jobs=1)


search = build_search(
    y_train=y_train,
    pipeline=build_pipeline(),
    parameter_grid=Param_distributions())

search.fit(X_train, y_train)
best_model = search.best_estimator_
```

For every country-period-outcome task, the procedure randomly evaluates 40 hyperparameter combinations using mean cross-validated ROC AUC. Shuffled stratified folds preserve the class distribution, while reducing the number of folds prevents a fold from containing no minority-class observations. Tasks with fewer than two minority-class training cases are skipped. Because `refit=True`, the selected pipeline is fitted again using the complete training set after tuning.

B.18 005 Random Forest (Model).ipynb - Cell 4 - Random Forest Classifier

The selected code below shows the Random Forest pipeline and the hyperparameters included in the random search.

```
from sklearn.ensemble import RandomForestClassifier
from imblearn.over_sampling import RandomOverSampler
from imblearn.pipeline import Pipeline

def build_pipeline():
    return Pipeline(steps=[
        ("ros", RandomOverSampler(
            random_state=RANDOM_STATE)),
        ("model", RandomForestClassifier(
            n_estimators=250,
            random_state=RANDOM_STATE,
            n_jobs=-1))
    ])
```

```
def Param_distributions():
    return {
        "model__max_depth":
            [5, 8, 10, 12, 15, 20, None],
        "model__min_samples_split":
            [2, 5, 10, 20, 30],
        "model__min_samples_leaf":
            [1, 2, 5, 10],
        "model__max_features":
            ["sqrt", "log2", 0.3, 0.5, 0.7, None],
        "model__bootstrap":
            [True]
    }
```

The classifier uses 250 trees, the fixed random seed `1948883` and all available processor cores. Random oversampling is applied within the pipeline so that it is performed separately inside each training fold. The search varies tree depth, the minimum numbers of observations required to split a node or form a leaf, and the number of predictors considered at each split.

B.19 006 XGBoost (Model).ipynb - Cell 4 - XGBoost Classifier

The selected code below shows the XGBoost pipeline and the hyperparameters included in the random search.

```
from xgboost import XGBClassifier
from imblearn.over_sampling import RandomOverSampler
from imblearn.pipeline import Pipeline

def Build_pipeline():
    return Pipeline(steps=[
        ("ros", RandomOverSampler(
            random_state=RANDOM_STATE)),
        ("model", XGBClassifier(
            n_estimators=250,
            objective="binary:logistic",
            eval_metric="logloss",
            tree_method="hist",
            random_state=RANDOM_STATE,
            n_jobs=-1))
    ])


def Param_distributions():
    return {
        "model__max_depth":
```

```
            [3, 4, 5, 6, 8, 10],
        "model__learning_rate":
            [0.01, 0.03, 0.05, 0.1, 0.2],
        "model__subsample":
            [0.6, 0.7, 0.8, 0.9, 1.0],
        "model__colsample_bytree":
            [0.5, 0.6, 0.7, 0.8, 0.9, 1.0],
        "model__min_child_weight":
            [1, 3, 5, 7],
        "model__gamma":
            [0, 0.01, 0.1, 0.3, 0.5, 1.0],
        "model__reg_alpha":
            [0, 0.01, 0.1, 1.0],
        "model__reg_lambda":
            [0.5, 1.0, 2.0, 5.0]
    }
```

The classifier uses 250 boosting trees with a binary logistic objective, log-loss evaluation and histogram-based tree construction. The random search tunes tree complexity, learning rate, row and column sampling, minimum child weight, split penalty and L1 and L2 regularisation. As in the Random Forest pipeline, oversampling is conducted within each training fold rather than before cross-validation.

B.20 005 Random Forest.ipynb and 006 XGBoost.ipynb - Cell 4 - Test ROC AUC

Cross-validated ROC AUC is used only for hyperparameter selection on the training data. Final performance is calculated once on the untouched test set using the predicted probability of the positive class. If the test target contains only one class, ROC AUC is stored as missing because it cannot be calculated.

```
from sklearn.metrics import roc_auc_score

def evaluate_tuned_model(search, X_test, y_test):
    best_pipeline = search.best_estimator_
    best_cv_auc = search.best_score_

    positive_probability = (
        best_pipeline.predict_proba(X_test)[:, 1]
    )

    if y_test.nunique() < 2:
        test_auc = np.nan
    else:
        test_auc = roc_auc_score(
```

```
            y_test,
            positive_probability
        )

    return {
        "best_cv_auc": best_cv_auc,
        "test_auc": test_auc,
        "best_parameters": search.best_params_
    }
```

B.21 005 Random Forest.ipynb and 006 XGBoost.ipynb - Cell4,5,7 - Feature importance

Feature importance is extracted from the fitted model step of the final refitted pipeline and exported as a complete table for every task. For comparison plots, before- and after-mandate tables are merged by feature, state dummy variables and explicit `N/A` indicators are excluded, and the ten variables with the largest importance in either period are retained. Each displayed variable is then assigned to a broad interpretation category.

```
def extract_feature_importance(
    fitted_pipeline,
    feature_names,
    output_path):
    fitted_model = fitted_pipeline.named_steps["model"]

    importance_table = pd.DataFrame({
        "feature": feature_names,
        "importance": fitted_model.feature_importances_
    }).sort_values(
        "importance",
        ascending=False)

    importance_table.to_csv(output_path, index=False)
    return importance_table


def prepare_before_after_plot(
    before_table,
    after_table,
    top_n=10):
    comparison = before_table.merge(
        after_table,
        on="feature",
        how="outer",
        suffixes=("_before", "_after")).fillna(0)
```

```
    keep = (
        ~comparison["feature"].str.startswith("State:", na=False)
        & ~comparison["feature"].str.contains(
            r"(^|: )N/A$",
            regex=True,
            na=False))

    comparison = comparison.loc[keep].copy()
    comparison["max_importance"] = comparison[
        ["importance_before", "importance_after"]].max(axis=1)

    return comparison.nlargest(
        top_n,
        "max_importance")


def feature_category(feature):
    text = feature.lower()

    if any(term in text for term in [
        "age", "gender", "employment",
        "education", "household"]):
        return "Demographics"

    if any(term in text for term in [
        "mask", "isolation", "handwash",
        "contact", "outings", "protective"]):
        return "Self-protective behaviours"

    if any(term in text for term in [
        "severity", "risk", "fear", "threat"]):
        return "Perception of illness threat"

    if any(term in text for term in [
        "health", "mental", "wellbeing",
        "comorbidity" ]):
        return "Health/mental-health/wellbeing"

    if "week" in text or "time" in text:
        return "Time"

    return "Other"


plot_table = prepare_before_after_plot(
    before_importance, after_importance)
```

```
plot_table["category"] = (
    plot_table["feature"].map(feature_category))
```

C Project structure

This appendix describes the computational project structure, software environment and execution order required to reproduce the data preparation, modelling and report outputs. All Python notebooks construct file paths from `Path.cwd()`. Consequently, Jupyter must be started from the project root directory, and the notebooks should remain in that directory when they are executed.

C.1 Computational environment

The metadata stored in the supplied notebooks identifies the computational kernel as `Python 3` and records Python version `3.13.14`. The analysis was implemented as Jupyter notebooks.

The statistical analysis is executed in Python rather than inside the R Markdown document. The final report is assembled separately using R Markdown. Rendering the report requires R, the `rmarkdown` and `knitr` packages, Pandoc, and a LaTeX distribution containing the `booktabs` and `float` packages.

C.2 Project directory structure

The required project layout is shown below. Raw data must be placed in the two folders under `Data` using the exact filenames expected by `001 Uniform column format.ipynb`. All folders under `Cleaned Data` and `Result` are generated by the notebooks.

```
Project root/
|-- Data/
|   |-- YouGov_Data/
|   |   |-- australia.csv
|   |   |-- brazil.csv
|   |   |-- canada.csv
|   |   |-- china.csv
|   |   |-- india.csv
|   |   |-- united-kingdom.csv
|   |   `-- united-states.csv
|   |
|   |-- codebook.xlsx
|   |
|   `-- OxCGRT_Data/
|       |-- OxCGRT_AUS_latest.csv
|       |-- OxCGRT_BRA_latest LEGACY.csv
|       |-- OxCGRT_CAN_latest.csv
```

```
|       |-- OxCGRT_CHN_latest.csv
|       |-- OxCGRT_GBR_latest.csv
|       |-- OxCGRT_IND_latest.csv
|       `-- OxCGRT_USA_latest.csv
|
|-- Cleaned Data/
|   |-- Uniform column format/
|   |   |-- YouGov_Australia.csv
|   |   |-- OxCGRT_Australia.csv
|   |   `-- corresponding YouGov and OxCGRT files for the other retained countries
|   |-- 002 Data Clean/
|   |   |-- cleaned_unencoded/
|   |   |-- cleaned_encoded/
|   |   `-- region_mandate_dates.csv
|   |-- 003 Column Names update/
|   |   |-- Columns update.csv
|   |   `-- <Country> (readable).csv
|   `-- 004 Split/
|       |-- Australia/
|       |-- Brazil/
|       |-- Canada/
|       |-- India/
|       |-- United_Kingdom/
|       `-- United_States/
|
|-- Result/
|   |-- Random Forest/
|   |   |-- random_forest_auc_results.csv
|   |   |-- random_forest_feature_importance_all.csv
|   |   `-- Figures/
|   `-- XGBoost/
|       |-- xgboost_auc_results.csv
|       |-- xgboost_feature_importance_all.csv
|       `-- Figures/
|
|-- 001 Uniform column format.ipynb
|-- 001_2 State mapping.ipynb
|-- 002 Data Clean.ipynb
|-- 003 Column Names update.ipynb
|-- 004 Split.ipynb
|-- 005 Random Forest (Model).ipynb
|-- 006 XGBoost (Model).ipynb
|-- 111 Dataset After Cleaning.ipynb
|-- 111 Why Mandate Thousold = 2.5.ipynb
`-- 111 AUC.ipynb
```

Within each country folder under `Cleaned Data/004 Split`, four modelling tasks are stored: face-mask wearing and overall protective behaviour, each before and after the mandate. Every task contains four files with the prefixes `X_train_`, `X_test_`, `y_train_` and `y_test_`.

C.3 Execution order

The notebooks should be run using **Restart Kernel and Run All** in the following order.

Step	Notebook	Function and main output
1	`001 Uniform column format.ipynb`	Audits the 14 raw CSV files, detects usable encodings, standardises required column names, excludes China from the common workflow and writes the retained YouGov and OxCGRT files to `Cleaned Data/Uniform column format`.
2	`001_2 State mapping.ipynb`	Harmonises YouGov state names with OxCGRT region names. The corrected CSV files overwrite the corresponding files in `Cleaned Data/Uniform column format`.
3	`002 Data Clean.ipynb`	Recodes variables, screens missingness, constructs the behavioural outcomes, assigns region-specific mandate periods, creates encoded and unencoded cleaned datasets, and writes `region_mandate_dates.csv`.
4	`003 Column Names update.ipynb`	Converts encoded column names into readable labels and writes one readable modelling dataset for each retained country.
5	`004 Split.ipynb`	Creates the four country-period-outcome tasks for each country and saves the 80/20 train–test files using random seed `1948883`.
6	`005 Random Forest (Model).ipynb`	Tunes and evaluates the Random Forest models, saves the AUC and complete feature-importance tables, and exports the Random Forest feature-importance figures.
7	`006 XGBoost (Model).ipynb`	Tunes and evaluates the XGBoost models, saves the AUC and complete feature-importance tables, and exports the XGBoost feature-importance figures.

Table 11: Required notebook execution order.

The three notebooks whose filenames begin with `111` are supplementary output and diagnostic notebooks rather than inputs to the main modelling pipeline:

- `111 Dataset After Cleaning.ipynb` can be run after `002 Data Clean.ipynb` to summarise row retention and the before/after-mandate sample sizes.
- `111 Why Mandate Thousold = 2.5.ipynb` can be run after the uniform policy files and `region_mandate_dates.csv` have been created to display the national policy trajectories and regional mandate dates.

- `111 AUC.ipynb` must be run after both model notebooks because it reads the Random Forest and XGBoost AUC result files.

These supplementary notebooks display plots and tables in Jupyter but do not contain commands that automatically save all displayed outputs. Figures required by the report must therefore be exported and placed in the project root using the filenames referenced by `PartB_Report.Rmd`.

C.4 Running the complete workflow

A complete reproduction can be carried out as follows:

1. Create the directory structure shown below and place all raw YouGov and OxCGRT CSV files in their required folders:

```
Project root/
|-- Data/
|   |-- YouGov_Data/
|   |   |-- australia.csv
|   |   |-- brazil.csv
|   |   |-- canada.csv
|   |   |-- china.csv
|   |   |-- india.csv
|   |   |-- united-kingdom.csv
|   |   `-- united-states.csv
|   |
|   |-- codebook.xlsx
|   |
|   `-- OxCGRT_Data/
|       |-- OxCGRT_AUS_latest.csv
|       |-- OxCGRT_BRA_latest LEGACY.csv
|       |-- OxCGRT_CAN_latest.csv
|       |-- OxCGRT_CHN_latest.csv
|       |-- OxCGRT_GBR_latest.csv
|       |-- OxCGRT_IND_latest.csv
|       `-- OxCGRT_USA_latest.csv
|
|-- 001 Uniform column format.ipynb
|-- 001_2 State mapping.ipynb
|-- 002 Data Clean.ipynb
|-- 003 Column Names update.ipynb
|-- 004 Split.ipynb
|-- 005 Random Forest (Model).ipynb
|-- 006 XGBoost (Model).ipynb
|-- 111 Dataset After Cleaning.ipynb
|-- 111 Why Mandate Thousold = 2.5.ipynb
`-- 111 AUC.ipynb
```

2. Start Jupyter from the project root so that `Path.cwd()` resolves to that directory.
3. Run notebooks `001`, `001_2`, `002`, `003` , `004`, `005`, `006`in sequence. (`005`, `006` The calculation of these two files takes a relatively long time, approximately 30 to 40 minutes.)
4. Run the relevant `111` notebooks to reproduce the diagnostic tables and plots used in the report.

Intermediate and final CSV files are written using `utf-8-sig` encoding. Existing generated files may be overwritten when the notebooks are run again. For a clean reproduction, the generated `Cleaned Data` and `Result` folders can be archived or removed before rerunning the workflow, while the raw files under `Data` should be preserved unchanged.

D Variable codebook

Table 12 summarises the main original and constructed variables used in the analysis. The table is not intended to reproduce the complete survey codebook; it provides definitions for the variables that are most important for understanding the outcomes, policy periods and feature-importance results.

Table 12: Selected variables used in the analysis.

Variable	Meaning	Role in the analysis
`i12_health_1`	Self-reported frequency of face-mask wearing, recorded from 1 (*Not at all*) to 5 (*Always*).	Used to construct the face-mask outcome.
Face Mask Wearing	Binary outcome equal to 1 when the recoded mask-wearing score was 4 or 5, and 0 otherwise.	Prediction outcome.
Non-Mask Protective Behavior Score	Median frequency across the 13 common protective-behaviour items excluding face-mask wearing.	Predictor in the face-mask tasks only.
Overall Protective Behavior Score	Median frequency across the 14 common protective-behaviour items, including face-mask wearing.	Used to construct the overall-behaviour outcome.
Overall Protective Behaviour	Binary outcome equal to 1 when the overall protective-behaviour score was at least 4, and 0 otherwise.	Prediction outcome.
`H6M_Facial Coverings`	OxCGRT indicator recording the strength of face-covering policy by date and, where available, subnational region.	Used to determine mandate dates.
Mandate period	Indicator showing whether a survey response occurred before or on/after the assigned regional mask-mandate date.	Defines the two policy periods.
Week	Relative survey-time variable constructed from consecutive 14-day intervals beginning on the first retained survey date in each country.	Time-related predictor.
Household size	Number of people in the respondent's household. Values of eight or more were coded as 8.	Demographic predictor.

Variable	Meaning	Role in the analysis
Comorbidity	Combined indicator showing whether the respondent reported at least one listed health condition.	Health-related predictor.
COVID Severity	Respondent's assessment of the seriousness of COVID-19, converted to an ordered numerical score.	Risk-perception predictor.
COVID Infection Risk	Respondent's perceived likelihood of becoming infected with COVID-19, converted to an ordered numerical score.	Risk-perception predictor.
Isolation Willingness	Respondent's reported willingness to self-isolate when required or advised.	Behavioural and attitudinal predictor.
Symptom Isolation	Respondent's reported intention or behaviour concerning self-isolation after developing possible COVID-19 symptoms.	Protective-behaviour predictor.
Physical Contact Count	Reported number of physical contacts, including contact with people outside the respondent's household.	Behavioural predictor.
Outings Count	Reported number of outings or activities outside the home during the relevant survey period.	Behavioural predictor.
Handwashing Count	Reported frequency or number of handwashing occasions.	Protective-behaviour predictor.
Employment	Respondent's employment status, represented by categorical indicators after one-hot encoding.	Demographic predictor.
Confidence in Government	Respondent's reported level of confidence in government during the pandemic.	Institutional-attitude predictor.
Government Response	Respondent's assessment of the government's response to COVID-19.	Institutional-attitude predictor.
PHQ Low Interest	Survey item measuring how often the respondent experienced little interest or pleasure in doing things.	Mental-health predictor.
Household COVID Test	Indicator describing whether a member of the respondent's household had received a COVID-19 test and the reported result or status.	Health-related predictor.

Categorical variables were converted into binary indicator columns using one-hot encoding. The displayed feature names therefore sometimes include both the original variable and a response category, such as *Employment: Retired* or *Isolation Willingness: Very Willing*.

More detailed definitions, response categories and descriptions of all original survey variables are provided in `codebook.xlsx`, which is available in the project GitHub repository (Zhao 2026).

E Protective-behaviour item composition

This appendix lists the survey items used to construct Face Mask Wearing, the Non-Mask Protective Behavior Score and Overall Protective Behaviour. The original item descriptions follow the Imperial College London/YouGov COVID-19 Behaviour Tracker codebook (Imperial College London and YouGov 2022).

All items used the same five response categories: `Not at all`, `Rarely`, `Sometimes`, `Frequently` and `Always`. These responses were recoded from 1 to 5 respectively. Scores of 4 or 5 represented frequent or consistent performance of the behaviour.

E.1 Face Mask Wearing

Face Mask Wearing was constructed from the single survey item shown in Table 13. The original response was retained as the Face Mask Wearing Score, and the binary outcome was equal to 1 when the score was at least 4.

Variable	Survey item
`i12_health_1`	Worn a face mask outside the home, for example when using public transport, visiting a supermarket or travelling on a main road.

Table 13: Survey item used to construct Face Mask Wearing.

E.2 Non-Mask Protective Behavior Score

The Non-Mask Protective Behavior Score was calculated as the median of the 13 items listed in Table 14. Complete responses to all 13 items were required. This score was retained as a predictor for the face-mask wearing tasks because it did not include the mask-wearing item.

Table 14: Survey items used to construct the Non-Mask Protective Behavior Score.

Variable	Survey item
`i12_health_2`	Washed hands with soap and water.
`i12_health_3`	Used hand sanitiser.
`i12_health_4`	Covered the nose and mouth when sneezing or coughing.
`i12_health_5`	Avoided contact with people who had symptoms or may have been exposed to coronavirus.
`i12_health_6`	Avoided going out in general.
`i12_health_7`	Avoided going to hospitals or other healthcare settings.
`i12_health_8`	Avoided taking public transport.
`i12_health_11`	Avoided having guests in the home.
`i12_health_12`	Avoided small social gatherings involving no more than two people.
`i12_health_13`	Avoided medium-sized social gatherings involving between three and ten people.

Variable	Survey item
i12_health_14	Avoided large social gatherings involving more than ten people.
i12_health_15	Avoided crowded areas.
i12_health_16	Avoided going to shops.

E.3 Overall Protective Behaviour

The Overall Protective Behavior Score was calculated as the median of all 14 items listed in Table 15. It combined the face-mask item with the 13 non-mask protective-behaviour items. Complete responses to all 14 items were required, and the binary outcome was equal to 1 when the median score was at least 4.

Table 15: Survey items used to construct Overall Protective Behaviour.

Variable	Survey item
i12_health_1	Worn a face mask outside the home, for example when using public transport, visiting a supermarket or travelling on a main road.
i12_health_2	Washed hands with soap and water.
i12_health_3	Used hand sanitiser.
i12_health_4	Covered the nose and mouth when sneezing or coughing.
i12_health_5	Avoided contact with people who had symptoms or may have been exposed to coronavirus.
i12_health_6	Avoided going out in general.
i12_health_7	Avoided going to hospitals or other healthcare settings.
i12_health_8	Avoided taking public transport.
i12_health_11	Avoided having guests in the home.
i12_health_12	Avoided small social gatherings involving no more than two people.
i12_health_13	Avoided medium-sized social gatherings involving between three and ten people.
i12_health_14	Avoided large social gatherings involving more than ten people.
i12_health_15	Avoided crowded areas.
i12_health_16	Avoided going to shops.